Normalization
Functional Dependencies
Normal Forms

a) **Conceptual Database Design**
   - The E-R Model

b) **Design principles and Methodologies**
   - Life cycle of databases within Information Systems
   - Quality of conceptual models

c) **Logical database Design**
   - Translating from the E-R model into the Relational Model

d) **Normalization**
   - Functional dependencies and normal forms

e) **Physical Database Design**
   - (hints to) Access structures, Indexing, Query plans

Normalized Logical Schemas

Objective: to obtain relational schemas that

- are anomaly free
- cause no information loss when computing joins

Two main concepts:

- **Functional dependencies**
- **Normal Forms**

Example

Someone thought it would be a good idea to have-it-all in one table, so as to be able to answer queries on people, budget and roles without worrying about the cost of computing joins…

EmpName	Salary	ProjName	Budget	Role
Rossi	2 K	beta	300	technician
Verdi	3 K	gamma	500	designer
Verdi	3 K	alpha	1500	designer
Neri	7 K	alpha	1500	prod.mgr
Neri	7 K	gamma	500	consultant
Neri	7 K	beta	300	consultant
Mori	6 K	beta	300	prod.mgr
Mori	6 K	alpha	1500	designer
Bianchi	6 K	alpha	1500	designer
Bianchi	6 K	beta	300	designer

Redundancy and Anomalies

1) **Redundancy:**

- The **Salary** of an employee is repeated more than once
- The **Budget** of a project is repreated more than once
- Attributes ProjName and EmpName are not unique per se
- the **key** is (EmpName, ProjName)

Redundancy and Anomalies

2) Any **UPDATE** of Salary or Budget would be problematic:

- Due to the redundancy, any update on the Salary of an employee should be applied to all the tuples relative to that employee
- Analogously, any update to the budget of a project should be applied to all tuples relative to that project

→ **UPDATE ANOMALY**

Updating one value (*i.e., what is* ***conceptually*** *one value*) requires updating more than one tuple

Redundancy and Anomalies

3) **DELETION** of an employee or project:

- Removing an employee may delete all data about the project (if they were the only employee associated to the project)
- Removing a project may accidentally delete all data about an employee (if working to that project only)
- We cannot store employees currently not working to any project, nor projects with no assigned employees

→ **DELETION ANOMALY**

Deleting a tuple may accidentally delete data that is not stored elsewhere

Redundancy and Anomalies

4) **INSERTION** of an employee:

- with (EmpName, ProjName) as key, it is impossible to insert an employee without a project (or a project without employees)
- Forcing such insertions would lead to null values in the key (either the project or the employee names)

→ **INSERTION ANOMALY**

Redundancy and Anomalies

- These situations are unlikely after an accurate and appropriate conceptual design
- Design errors may lead to anomalies
- More often, rather than happening due to blatant design errors, these anomalies may occur in legacy systems, where the schema has been subject to many layered modifications and adjustments, applied to adapt the system to changes that would have required more careful re-design steps

Functional Dependencies: definition

Key constraints (as we studied them) are part of a broader family of constraints, known as **functional dependencies**

- Given two sets of attributes X and Y in a relation R
- We say that X determines Y (or Y is functionally dependent on X), that we write in the form of an implication $X \rightarrow Y$, if and only if:

For each couple of distinct tuples t_1 , t_2 *in R,*

if $t_1[X] = t_2[X]$, *then necessarily* $t_1[Y] = t_2[Y]$

- As a consequence of the definition, every attribute (or set of attributes) of R is functionally dependent on any key of R
- X and Y are sometimes referred to as the *LHS* and *RHS* of the dependency, or the *Determinant* and the *Dependent*, respectively

In the example

- The primary key (obviously) determines all the other attributes:
  - EmpName, ProjName $\rightarrow$ Salary
  - EmpName, ProjName $\rightarrow$ Budget
  - EmpName, ProjName $\rightarrow$ Role

Compactly expressed as EmpName, ProjName $\rightarrow$ Salary, Budget, Role

- However, we also observe these other two (more specific) dependencies, that "override and imply" the first two :
  - EmpName $\rightarrow$ Salary
  - ProjName $\rightarrow$ Budget

- Let's focus on the three "free" dependencies

Some dependencies cause anomalies

EmpName, ProjName → Role

Accounts for the fact that each employee can only play one role in each project

EmpName → Salary

Means that each employee has a unique salary (independently of the project)

ProjName → Budget

Accounts for the fact that the budget of a project is a characteristic of the project, independently of any other attribute

Why do the first two dependencies cause anomalies, while the third doesn't?

→ The LHS of EN, PN → Role is the key (no anomalies)

→ The LHS of EN → Salary and PN → Budget is not the key (anomalies)

Boyce-Codd Normal Form (BCNF)

A relation R is in BCNF if
for every dependency $X \rightarrow Y$ in R X is a superkey for R
(i.e., X contains a key K for R)

In words, this means that a relation, in order to be in BCNF, is required to only contain attributes that are homogeneous and specific to the concept represented by the relation

- all non-key attributes express properties directly associated to the key, properties that are determined by the «identity» of the items in the relation

What can we do if one of our relations is **not** in BCNF?

Decomposition of a non-BCNF relation

- A non-BCNF relation can be decomposed in two or more relations that are in BCNF

- This **decomposition** can be applied by projecting the relation on the restricted schemas of the functional dependencies, so that each dependency generates **a new**, separate **relation <u>with the LHS as key</u>**

- In the example:

from EN → Salary *we derive* EMPLOYEE(<u>EmpName</u>, Salary)
from PN → Budget *we derive* PROJECT(<u>ProjName</u>, Budget)
from EN, PN → Role *we derive* WORKS_TO(<u>EmpName, ProjName</u>, Role)

Example of decomposition

EMPLOYEE

EmpName	Salary
Rossi	2 K
Verdi	3 K
Neri	7 K
Mori	6 K
Bianchi	6 K

PROJECT

ProjName	Budget
beta	300
gamma	500
alpha	1500

WORKS_TO

EmpName	ProjName	Role
Rossi	beta	technician
Verdi	gamma	designer
Verdi	alpha	designer
Neri	alpha	prod.mgr
Neri	gamma	consultant
Neri	beta	consultant
Mori	beta	prod.mgr
Mori	alpha	designer
Bianchi	alpha	designer
Bianchi	beta	designer

Join of de-composed tables for reconstruction

The three new relations are normalized, as they are now in BCNF.

The original, non-decomposed and non-normalized relation can be reconstructed as:

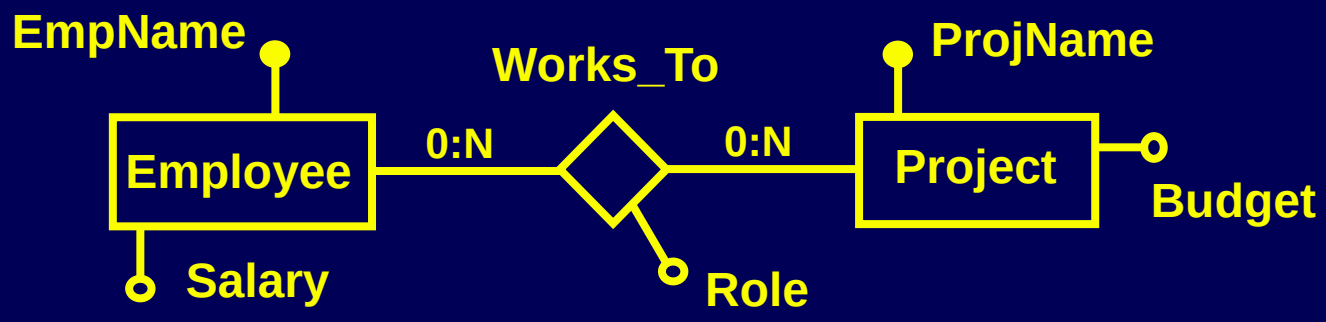
```
select *
from EMPLOYEE E, PROJECT P, WORKS_TO W
where  E.EmpName  = W.EmpName
    and  W.ProjName = P.ProjName
```

Or, in this case, also as:

```
select *
from EMPLOYEE natural join WORKS_TO
       natural join PROJECT
```

Non-lossy decompositions

- When it is possible to reconstruct the original relation with joins, we say that the decomposition is **correct** and **non-lossy** (or lossless).
- A good conceptual design typically avoids these problems ab-initio. Indeed, we would have probably modeled our example as:



Example of a lossy decomposition

EmpName	ProjName	City
Rossi	beta	milano
Verdi	gamma	torino
Verdi	alpha	torino
Bianchi	delta	milano
Neri	gamma	torino

EmpName → City
ProjName → City

Decomp

EmpName	City
Rossi	milano
Verdi	torino
Bianchi	milano
Neri	torino

Name → City

ProjName	City
beta	milano
gamma	torino
alpha	torino
delta	milano

Project → City

If we attempt to rebuild the initial table with a join on City we obtain:

join

EmpName	ProjName	City
Rossi	beta	milano
Rossi	*delta*	*milano*
Verdi	gamma	torino
Verdi	alpha	torino
Bianchi	*beta*	*milano*
Bianchi	delta	milano
Neri	gamma	torino
Neri	*alpha*	*torino*

That's not the original relation!

Quality of a decomposition

- Decompositions should always be non-lossy and should also preserve all FDs:

  ✓ A non-lossy decomposition guarantees that all the information in the original relations can be reconstructed starting from the de-composed relations

  ✓ The preservation of FDs guarantees that the de-composed relations have the same expressivity, i.e. they enforce the same integrity constraints as the original relations (and therefore prevent anomalies from occurring)

BCNF: is it always possible to obtain it?

Manager	Project	City
Brown	Mars	Chicago
Green	Jupiter	London
Green	Mars	London
Hoskins	Saturn	London
Hoskins	Venus	London

With the following FDs:

Manager → City

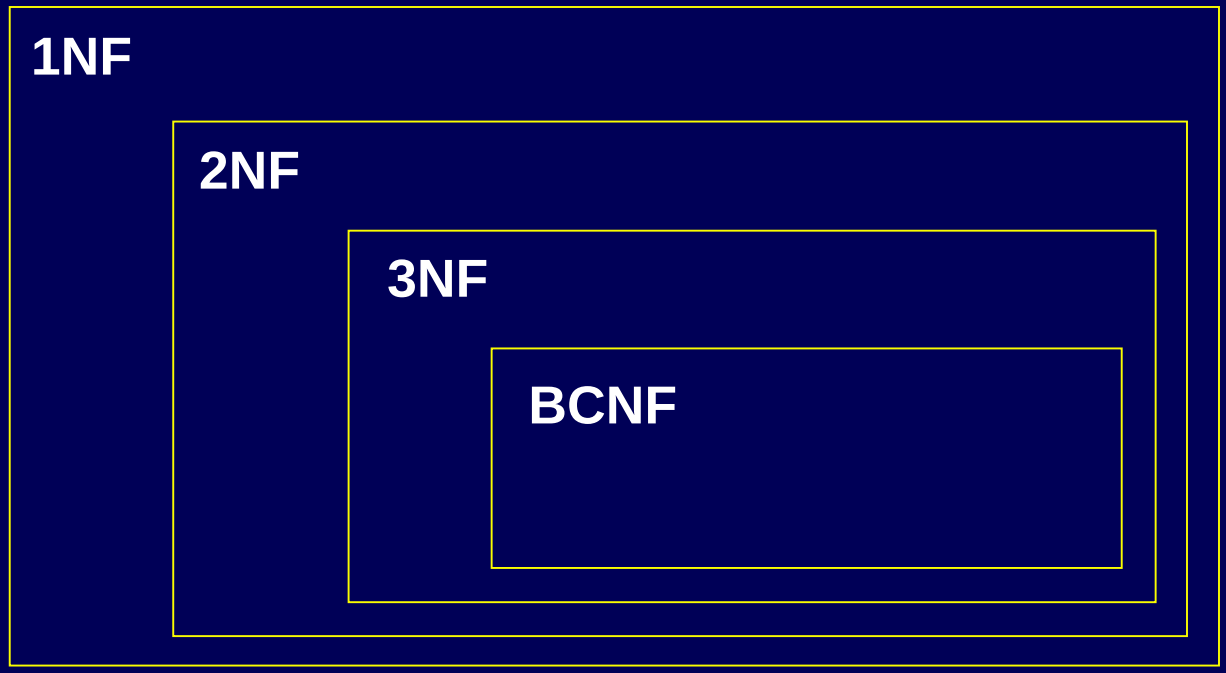
Project, City → Manager

BCNF: is it always possible to obtain it?

- The relation is not in BCNF because the determinant of **M** → **C** is not a superkey
- Moreover, no good decomposition is possible, because **P, C** → **M** involves all attributes, and cannot be preserved by any de-composition

→ No, it is not always possible to get a BCNF version of a schema!

Overview of several normal forms



Third Normal Form (3NF)

A relation R is in **Third Normal Form (3NF)** if, for every functional dependency **X** $\rightarrow$ **Y** defined on R, at least one of the following holds:

- **X** contains a key of R
- **Y** is contained in at least one key of R

It is reasonable to expect that a good schema is at least in 3NF

i.e., we expect you to design your schemas accordingly

3NF and BCNF coincide if R only has a one-attribute K

BCNF and 3NF

- The previous relation is **not in BCNF** but **it is in 3NF**:
  - In **P, C** → **M** the determinant is the key of the relation
  - In **M** → **C** the dependent is part of the key
- So, 3NF is less restrictive than the BCNF (and guarantees for a weaker quality)
- However, it can always be obtained preserving all FDs
  - → *there are algorithms for the general case*

Other well known normal forms

1NF : a relation is in 1NF iff all its attributes are defined on a domain with only atomic values (it is implied by the Relational Model, as it is one of its distinctive features)

2NF : a relation is in 2NF iff:

- it is in 1NF, and
- all its non-key attributes functionally depend on the **entire key** (meaning that no partial dependencies are allowed)

3NF (alternative definition) : a relation is in 3NF iff:

- It is in 2NF
- It does not contain transitive dependencies from the key to non-key attributes (i.e., all non-key attributes are directly determined by the key)

To normalize or Not to normalize…

- Normalization avoids some **anomalies**, but at the cost of **heavier computations**, as more joins may be required to answer the same queries
- The **update frequency** of the dataset is crucial to determine which alternative is preferable
- Non-normalized relations have some **redundancy** that shall be measured and carefully documented, to assess
  - extra memory occupancy
  - extra cost when performing updates

In summary…

- A normal form is a set of properties to guarantee a «quality level» of a schema, in the form of absencce of specific flaws
- A non-normalized relation is prone to **redundancy** and undesired **anomalies** when subject to modifications
- The most relevant Normal Forms (3NF and BCNF) are defined via the notion of functional dependency (FD)
- A schema can be normalized by decomposing non-normalized relations in multiple sub-relations
- A decomposition is useful and interesting if it is non-lossy (i.e., the initial relation can be reconstructed exactly as it was)
- It is also desirable that the decomposition preserves all FDs, so as to guarantee that all the constraints are enforced

Physical Database Design
(hints)

a) **Conceptual Database Design**
   - The E-R Model

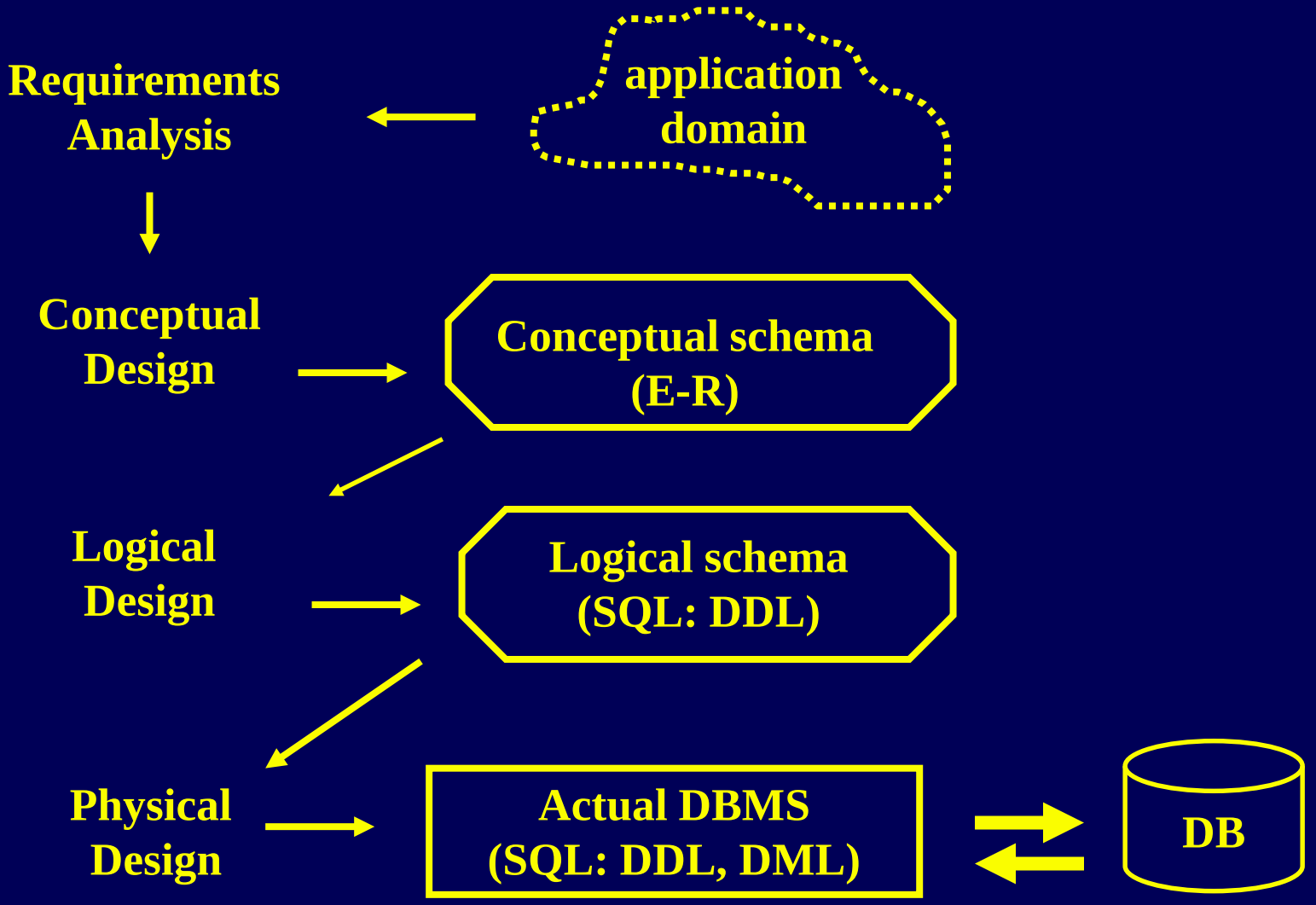
b) **Design principles and Methodologies**
   - Life cycle of databases within Information Systems
   - Quality of conceptual models

c) **Logical database Design**
   - Translating from the E-R model into the Relational Model

d) **Normalization**
   - Functional dependencies and normal forms

e) **Physical Database Design**
   - (hints to) Access structures, Indexing, Query plans

Phases of the Database Design Process



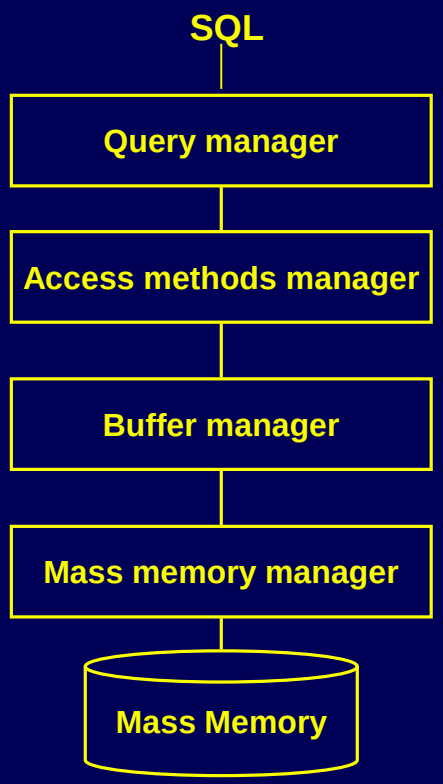
Design Phases

- **1. Conceptual design**
  - Translates the summary of requirements into a formal model: a conceptual schema that shall be independent of any DBMS choice (for us: an E-R model)
- **2. Logical design**
  - Translates the conceptual schema into a logical schema, expressed in one of the many logical models (Hierarchical, Relational, Object-Oriented, XML, …)
- **3. Physical design**
  - Completes the logical schema with whatever detail is required to assure efficiency (tuning of physical access structures, indexes, …)
  - This is widely dependent of the specific DBMS of choice

Dependencies: DATA MODEL and DBMS

	Dependent of the **DATA MODEL**	Dependent of the **DBMS**
Conceptual design	**No**	**No**
Logical design	***Yes***	**No**
Physical design	***Yes***	***Yes***

Queries and memories



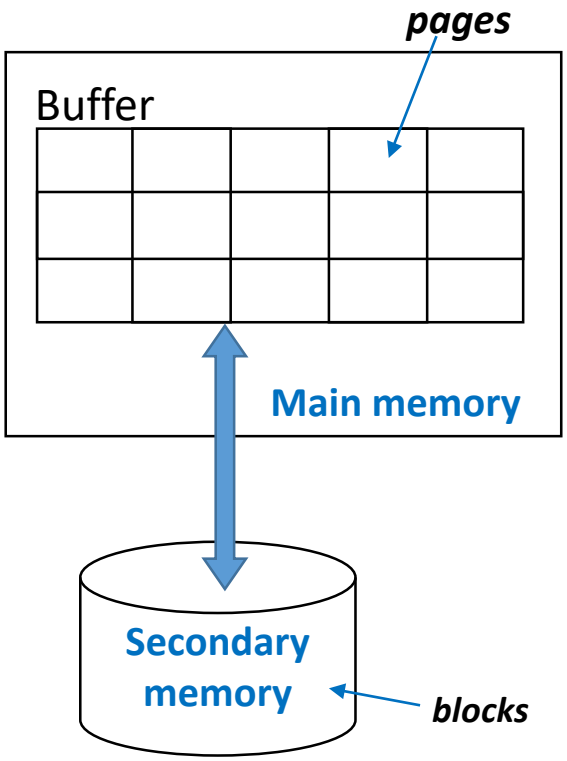
The buffer manager has the responsibility to keep part of the the DB in main memory:

- To achieve *efficiency*
- Taking care of *reliability* and *consistency*

DATA ACCESS and COST MODEL

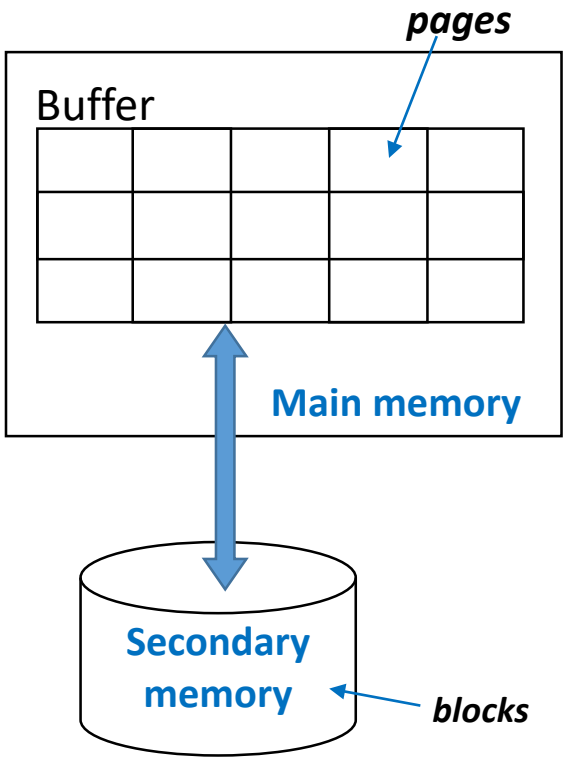
Main and Secondary memory

- Databases must be stored (mainly) in files onto secondary memory for two reasons:
  - size
  - persistence
- Data stored in secondary memory can only be used if first transferred to main memory
- Page: unit of storage in main memory
- Block: unit of storage in secondary memory



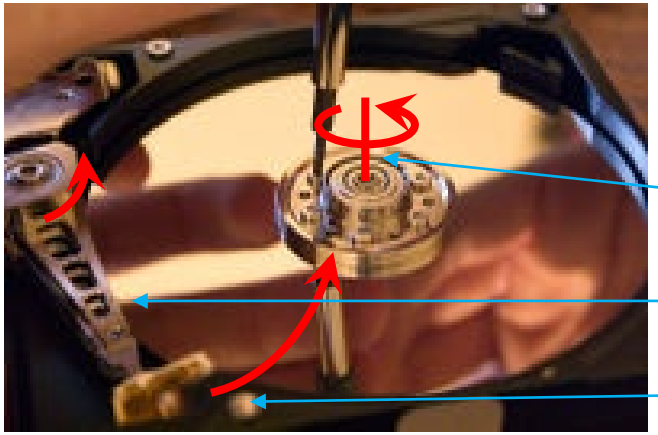
Pages and blocks

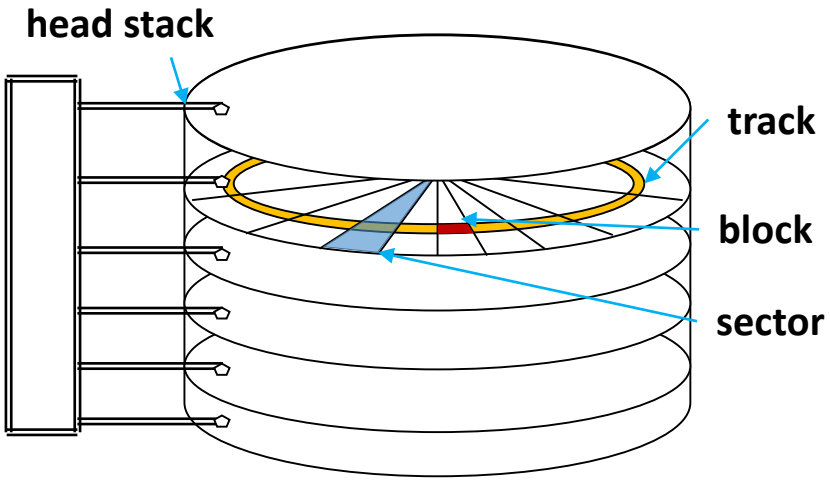
- Secondary memory devices are organized in blocks of (usually) fixed length
  - order of magnitude: a few KBytes
- We make the *assumption* that the size of a page equals the size of a block
  - In real systems a page may contain several blocks
- I/O operation: moving a block from secondary to main memory and vice-versa



A mechanical hard drive (Winchester)

- Several **disks** piled and rotating at constant angular speed
- A **head stack** mounted onto an arm moves radially in order to reach the **tracks** at varius distances from the rotation axis (**spindle**)
- A particular **sector** is «reached» by waiting for it to pass under one of the heads
- Several **blocks** can be reached at the same time (as many as the number of heads/disks)





→ in depth in the course
Computing Infrastructures, semester 2

Main vs. Secondary memory access time

- Secondary memory access:
  - seek time (8-12ms) - head positioning on the correct track
  - latency time (2-8ms) - disc rotation on the correct sector
  - transfer time (~1ms) - data transfer
- The cost of an access to secondary memory is 4 orders of magnitude higher than that to main memory
- In "I/O bound" applications the cost exclusively depends on the number of accesses to secondary memory
- **Cost of a query = #blocks** to be moved to execute a query

DBMS and file system

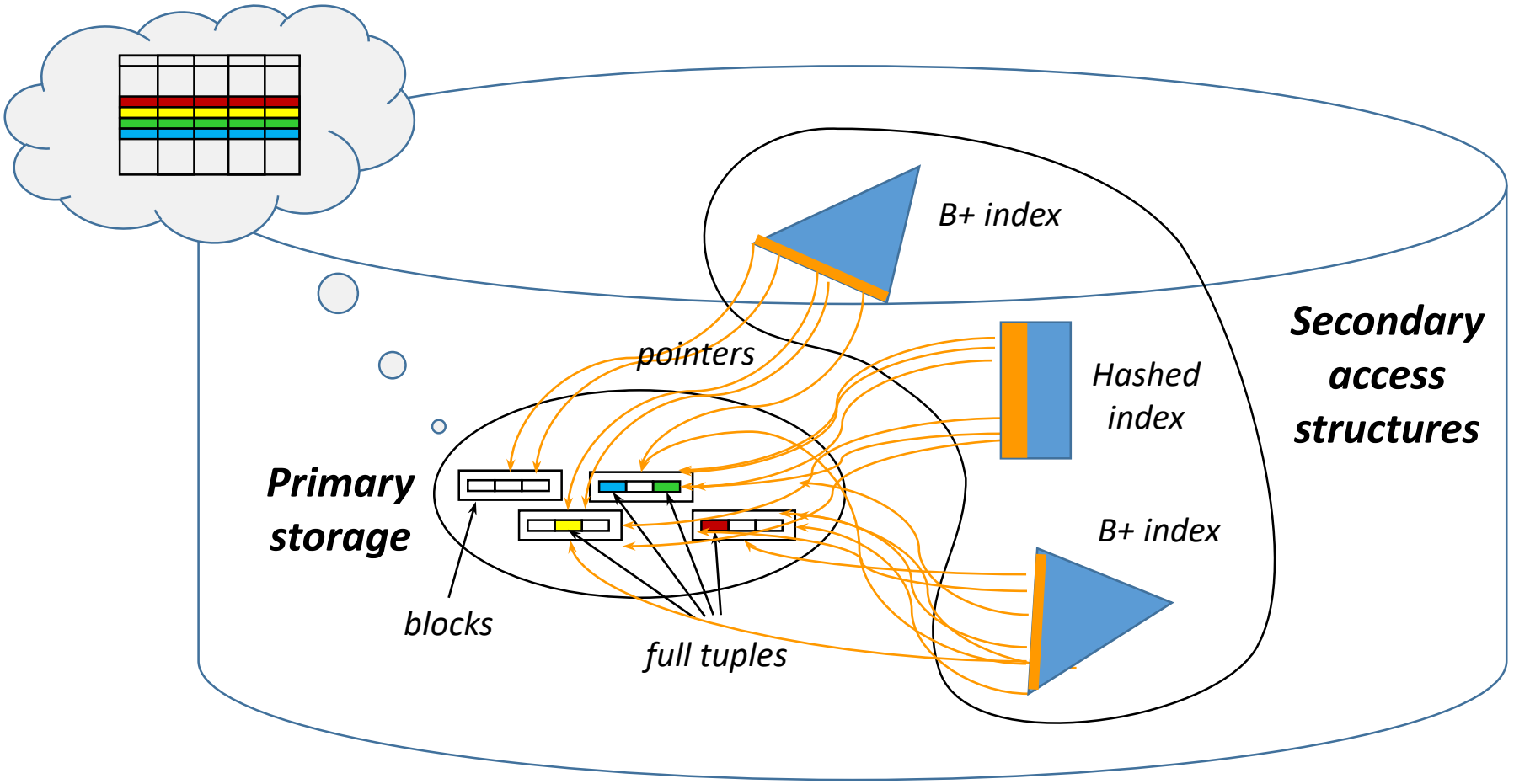
- File System (FS): component of the OS which manages access to secondary memory
- DBMSs make limited use of FS functionalities (create/delete files, read/write blocks)
- The **DBMS directly manages the file organization**, both in terms of the distribution of records within blocks and with respect to the internal structure of each block
  - A DBMS may also control the physical allocation of blocks onto the disk (for faster sequential reads, fine grain control of write operation execution time, reliability, etc)

PHYSICAL ACCESS STRUCTURES

Physical access structures

- Each DBMS has a distinctive and limited set of access methods
  - **Access methods**: software modules that provide data access and manipulation (store and retrieve) primitives **for each physical data structure**
- Access methods have their own data structures to organize data
  - Each table is stored into exactly one **primary** physical data structure, and
  - may have one or many optional **secondary** access structures

Primary and Secondary access structures



Physical access structures

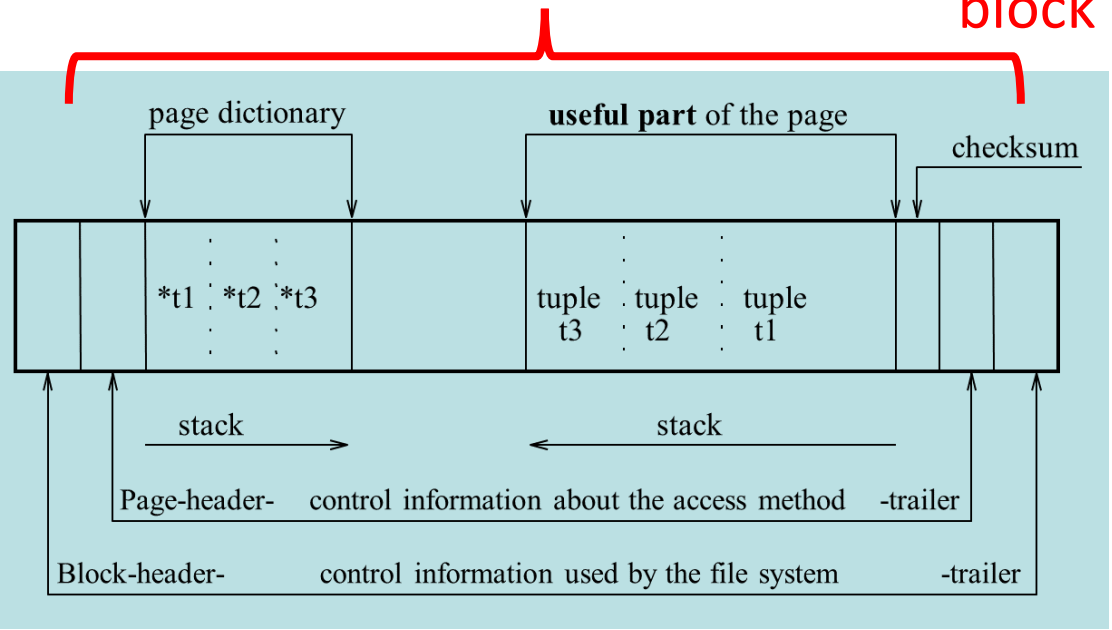
- **Primary structure**: it contains all the **tuples** of a table
  - Main purpose: to store the table content
- **Secondary structures**: are used to **index** primary structures, and only contain the values of some fields, interleaved with pointers to the blocks of the primary structure
  - Main purpose: to speed up the search for specific tuples, according to some search criterion
- Three main types of data access structures:
  - **Sequential** structures
  - **Hash-based** structures
  - **Tree-based** structures

Blocks and tuples

- **Blocks**: the "physical" components of files
- **Tuples**: the "logical" components of tables
  - The size of a block is typically **fixed** and depends on the file system and on how the disk is formatted
  - The size of a tuple (also called record) depends on the database design and is typically **variable** within a file
    - optional (possibly null) values, varchar (or other types of non-fixed size) attributes

Organization of tuples in blocks

Block for sequential and *hash-based* methods



- Block header and trailer with control information used by the file system
- Page header and trailer with control information about the access method
- A page dictionary, which contains pointers (offset table) to each elementary item of useful data (e.g., tuple) contained in the page
- A useful part, which contains the **data** (e.g., tuples)
  - In general, page dictionaries and useful data grow as stacks in opposite directions
- A checksum, to detect corrupted data

Sequential structures

- Sequential arrangement of tuples in the secondary memory (access to next element is supported)
- Blocks can be contiguous on disk or sparse
- Two cases:
  - Entry-sequenced organization: sequence of tuples dictated by their order of entry
  - Sequentially-ordered organization: tuples ordered according to the value of a key (one or more attributes)

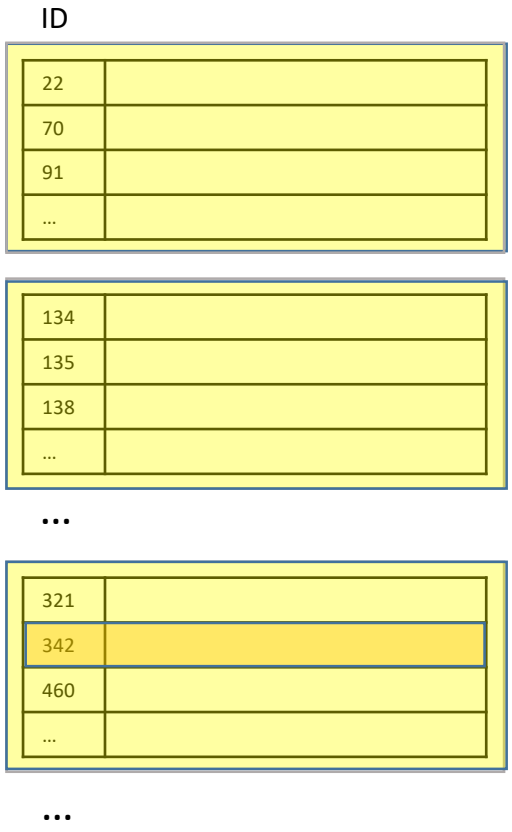
Entry-sequenced sequential (a.k.a. heap) structure

- Effective for
  - Insertion, which does not require shifting
  - Space occupancy, as it uses all the blocks available for files and all the space within the blocks
  - Sequential reading and writing (**select * from T**)
    - Especially if the disk blocks are contiguous (seek & latency times reduced)
    - Only if all (or most of) the file is to be accessed
- Non-optimal for
  - Searching specific data units (**select * from T where…**)
    - may require scanning the whole structure
    - But **with indexes** can be used efficiently
  - Updates that increase the size of a tuple (“shifts” required)
    - Shift may require storage in another block
    - Alternative approach: delete old version and insert new one

Sequentially-ordered sequential structure

- Tuples are sorted based on the value of a key field
- Effective for
  - Range queries that retrieve tuples having key in an interval
  - Order by and group by queries exploiting the order
- Problem: insertions & updates that increase the size
  - Reordering of the tuples within the block, if space allows
  - Techniques to avoid global reordering:
    - Differential files + periodic merging (example: paper yellow pages)
    - Free space in the block at loading → 'local reordering' operations
    - Overflow file: new tuples are inserted into blocks linked to form an overflow chain (general principle used also in other organizations)

Example of query on an entry-sequenced sequential structure




```
SELECT *   FROM  Student    WHERE Student.ID = '342'
```

#I/O operations = # of read blocks

Comparison

	Entry-sequenced	Sequentially-ordered
INSERT	Efficient	Not efficient
UPDATE	Efficient *(if data size increases → delete + insert the new version) *modulo the finding of data	Not efficient if data size increases
DELETE	"Invalid"	"Invalid"
TUPLE SIZE	Fixed or variable	Fixed or variable
SELECT * FROM T WHERE key …	Not efficient	More efficient

Entry-sequenced organization is the most common solution, but **PAIRED** with secondary access structures (indexes)

Hash-based access structures

- Efficient associative access to data, based on the value of a key
- A hash-based structure has N_B buckets (with N_B << number of data items)
  - A bucket is a unit of storage, typically of the size of 1 block
  - Often buckets are stored adjacent in the file
- A hash function maps the key field to a value between 0 and N_B–1
  - This value is interpreted as the index of a bucket in the hash structure (hash table)
- Efficient for
  - Tables with small size and (almost) static content
  - Point queries: queries with equality predicates on the key
- Inefficient for
  - Range queries and full table queries
  - Tables with very dynamic content

How hash-based structures work

hash(fileId, Key): BlockId

- The implementation consists of two parts
  - folding, transforms the key values (e.g., strings) so that they become positive integer values, uniformly distributed over a large range
  - hashing transforms the positive number into a number between 0 and N_B–1, to identify the right bucket for the tuple

Hash function
h(K) = K mod 10

KEY K=981 ⟶ h(981): bucket 1

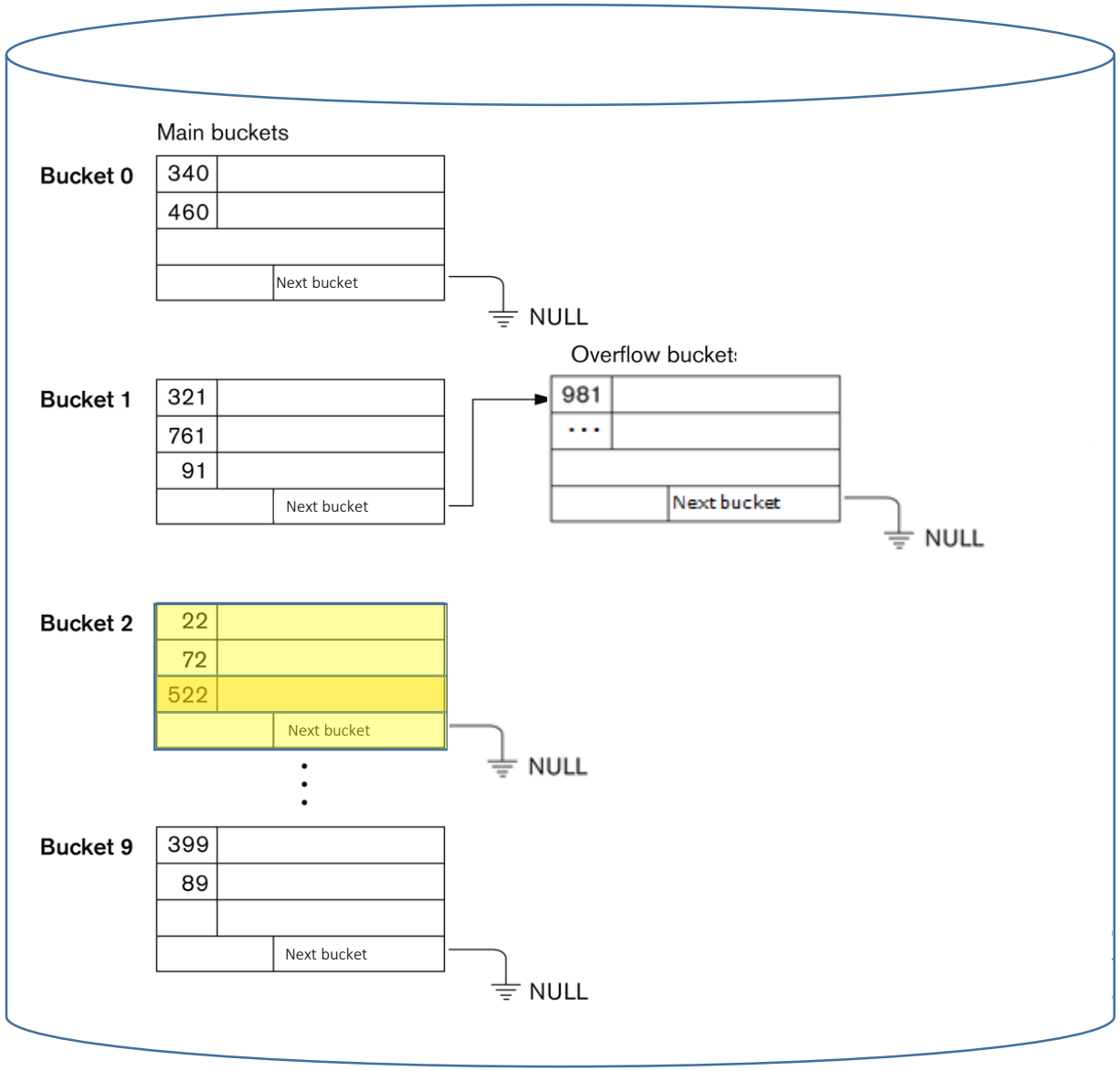
Hash table for primary storage

Hash function
h(K) = K mod 10

```
SELECT *
FROM  Student
WHERE Student.ID = '522'
```

h(522) = 2

#I/O operations = 1



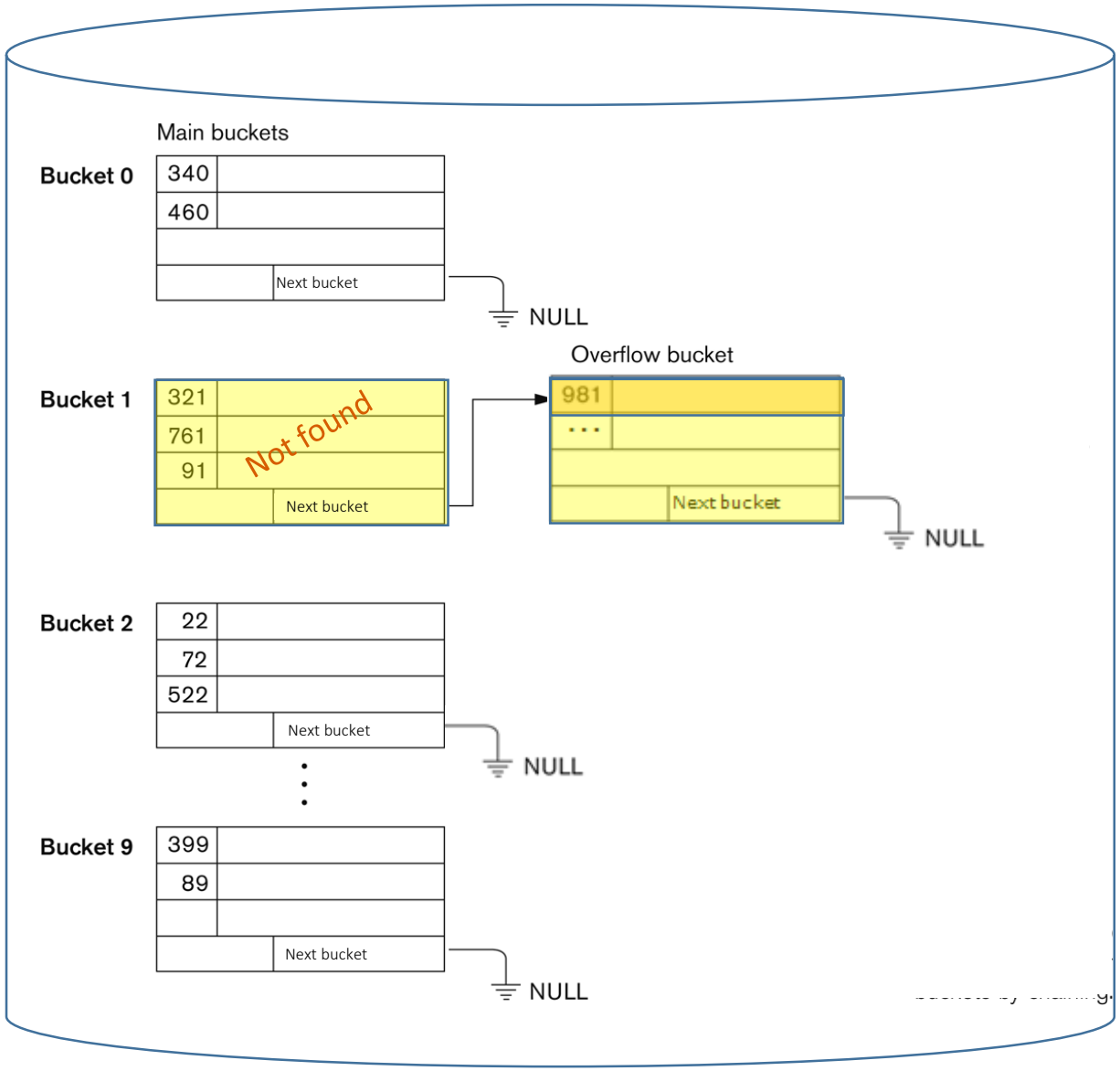
Hash table for primary storage

Hash function
h(K) = K mod 10

```
SELECT *
FROM  Student
WHERE Student.ID = '981'
```

h(981) = 1

#I/O operations = 2

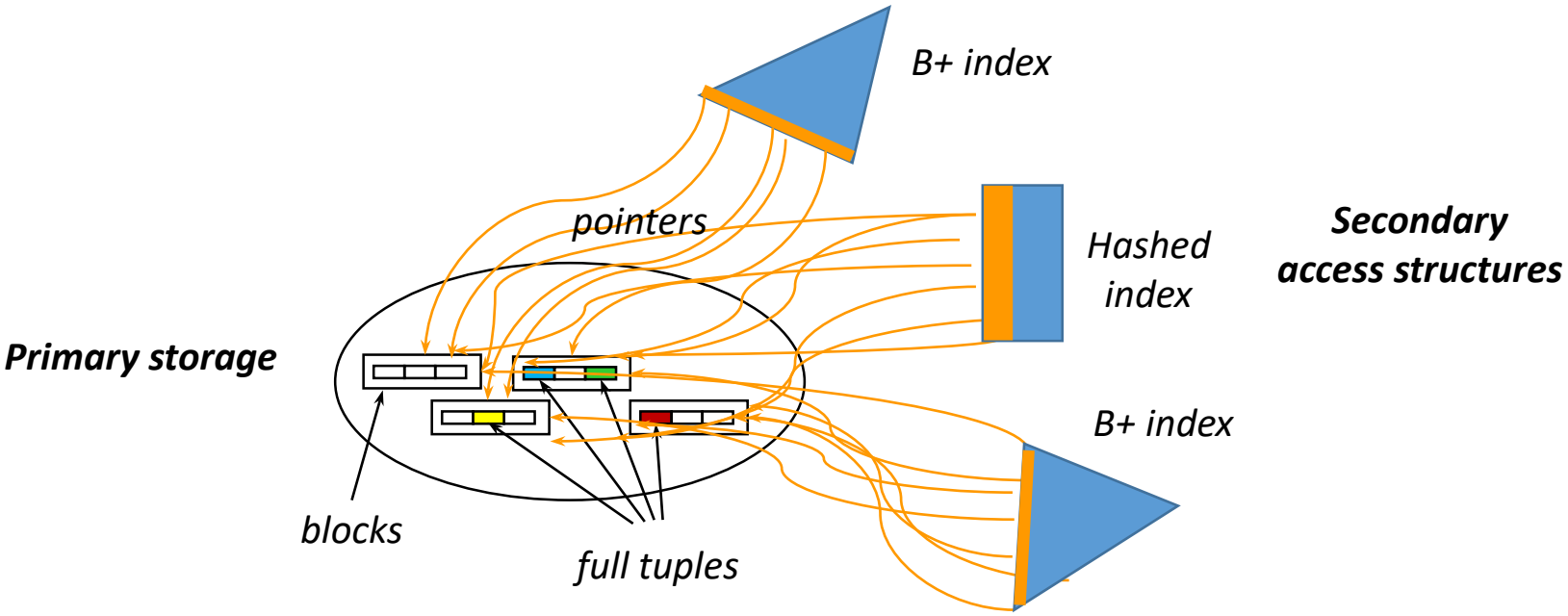


Hash-based primary storage - collisions

- How many accesses are needed to find the tuple corresponding to a given key?
- Most of the times 1 access, sometimes 2 accesses or more
- We can estimate the cost of accessing the tuple by considering the average length of the overflow chain
  - The more collisions, the longer the overflow chains…

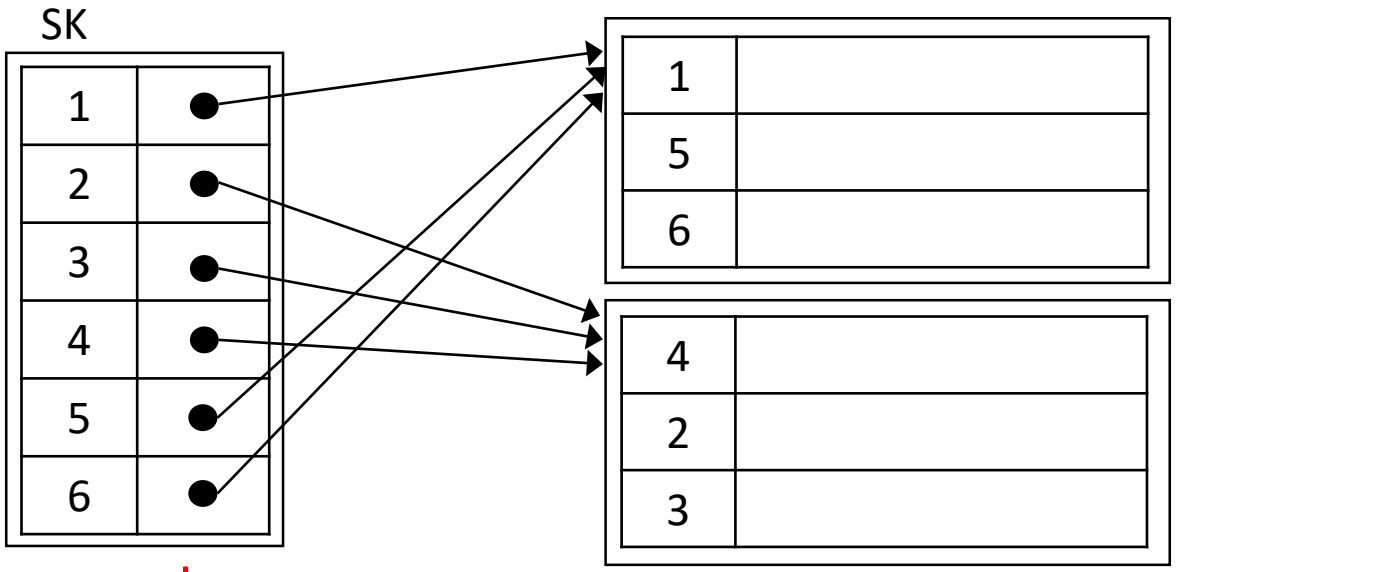
Indexes

- Data structures that help efficiently retrieve tuples on the basis of a **search key (SK)**
- They contain records of the form [search key, pointer to block]
- Index entries are sorted w.r.t. the key (but for hash-based indices, see later)
- The index concept: analytic index of a book → pair (term - page number) alphabetically ordered at the end of a book



Secondary index

- Secondary index:
  - The search key specifies an order different from the sequential order of the file **(SK != OK)**
  - Multiple secondary indexes can be defined for the same table, on different search keys
  - It is necessarily **dense**, because tuples with contiguous values of the key can be distributed in different blocks



A search key is not a primary key!

- Don’t get confused:
- **Primary key**: set of attributes that uniquely identify a tuple (minimal, unique, not null)
  - Does not imply access path
  - In SQL “PRIMARY KEY” defines a **constraint**
  - Implemented by means of an index
- **Search key**: set of attributes used in an index to speed up tuple location
  - Physical implementation of access structures
  - Defines a common access path
    - Each search key value is associated with one or more pointers
    - May be unique (one pointer) or not unique (multiple pointers or pointer to block)

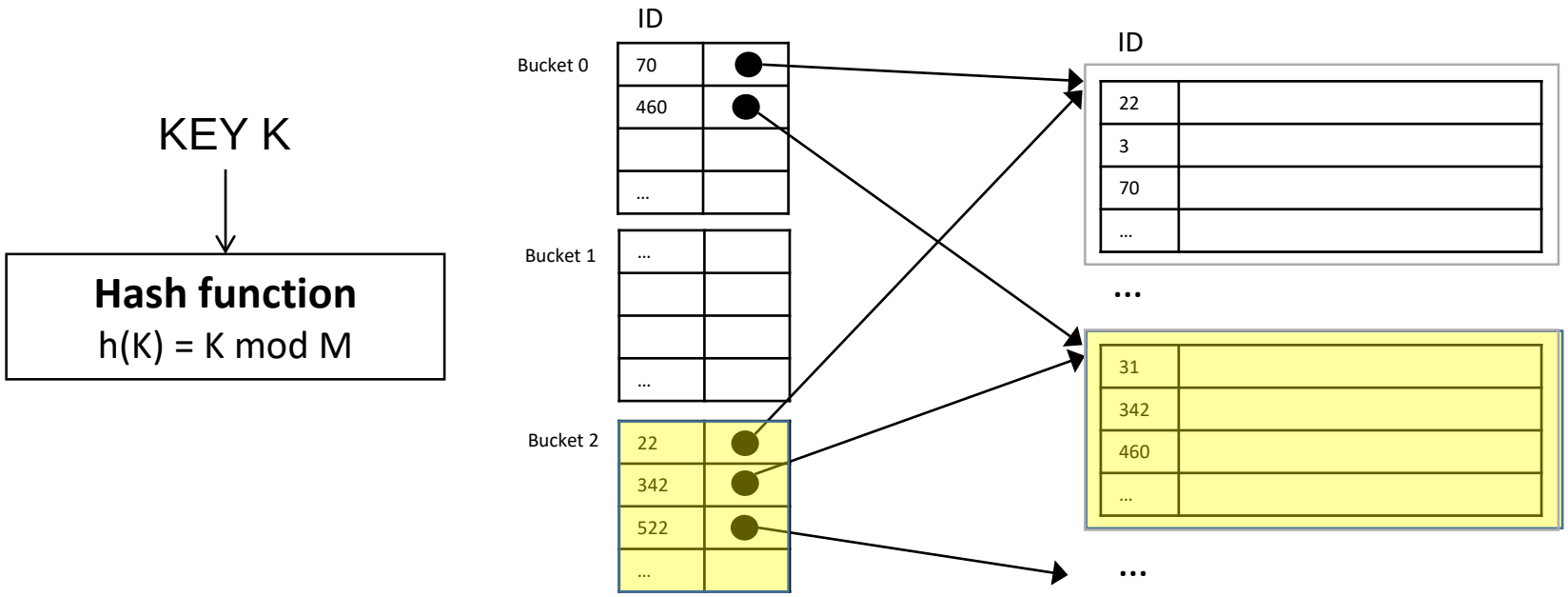
Pros & cons of indexes

- Smaller than primary data structures, can be loaded in main memory
- Support point/range queries and sorted scans efficiently
  - But less efficient than hash primary structures for point queries
- BUT: adding indexes to tables means that the DBMS has to **update the index** too after an insert update or delete operation
- Indexes do not come for free, they may slow down data changing operations

Using hash-based structures as indexes

- Hash-based structures can be used for **secondary** indexes
- Shaped and managed exactly like a hash-based primary structure, but
  - instead of the tuples, the buckets only contain key values and pointers

Hash-based index



…

Hash index
(in secondary memory)

Data – for example, in an entry-sequenced structure

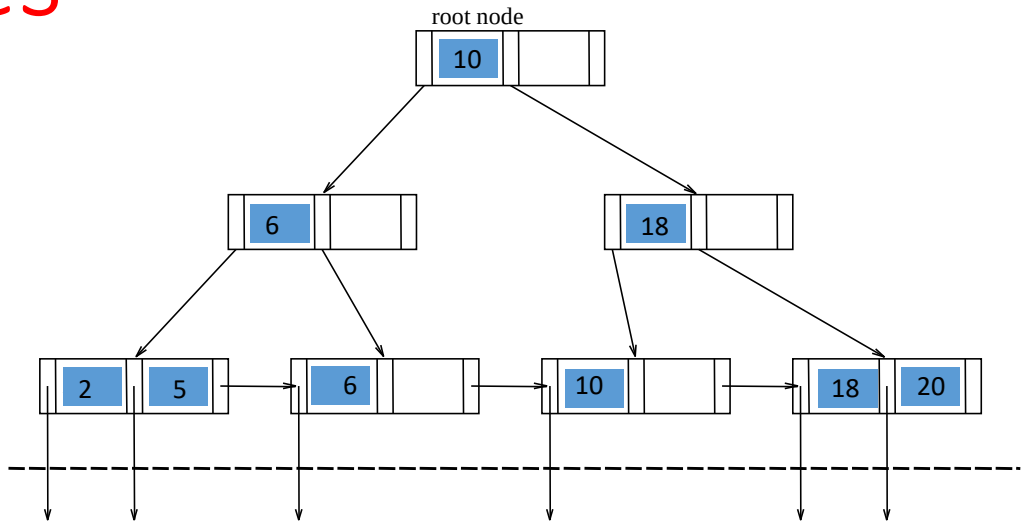
```
SELECT *   FROM  Student    WHERE Student.ID = '342'
```

#I/O operations = 2 (without overflow chains)

Tree-based structures

- Most frequently used in relational DBMSs for secondary (index) structures
  - SQL indexes are implemented in this way
- Support associative access based on the value of a key search field
- **Balanced trees (B trees)**: the lengths of the paths from the root node to the leaf nodes are all equal
  - Balancing improves performance
- Two main file organizations:
  - **B trees**: key values stored *also in internal nodes*
  - **B+ trees** (most used): all key values stored *in leaf nodes*

B+ tree structures



- Evolution from B trees
- Multi-level index
  - one root node
  - several intermediate nodes
  - several leaf nodes
- Each node is stored in a block
- In general, each node has a large number of descendants (fan out or **degree**), and therefore the majority of blocks are leaf nodes
- The fan out depends on the size of the block, the size of key values and the size of pointers

Examples of insert/delete in a B+ tree (visualization tool)

Indexes in SQL

- Syntax in SQL:

```
create [unique] index IndexName on
TableName(AttributeList)

drop index IndexName
```

- Some examples:
  - Oracle create index
  - MySQL create index

Indexes in SQL

- Every table should have:
  - A suitable primary storage, possibly sequentially ordered (normally on unique values, typically the primary key)
  - Several secondary indexes, both unique and not unique, on the attributes most used for selections and joins
- Secondary structures are progressively added, checking that the system actually uses them

Some guidelines for choosing indexes

1. Do not index small tables
2. Index Primary Key of a table if it is not a key of the primary file organization
   - Some DBMS automatically create unique indexes on primary keys and unique keys
3. Add a secondary index to any column that is heavily used as a secondary key
4. Add a secondary index to a Foreign Key if it is frequently accessed
5. Add secondary indexes on columns that are involved in: selection or join criteria; ORDER BY; GROUP BY; other operations involving sorting (such as UNION or DISTINCT)
6. Avoid indexing a column or table that is frequently updated
7. Avoid indexing a column if most queries retrieve a significant proportion of the records in the table
8. Avoid indexing columns that consist of long character strings

INTRODUCTION TO OPTIMIZATION

COST-BASED OPTIMIZATION

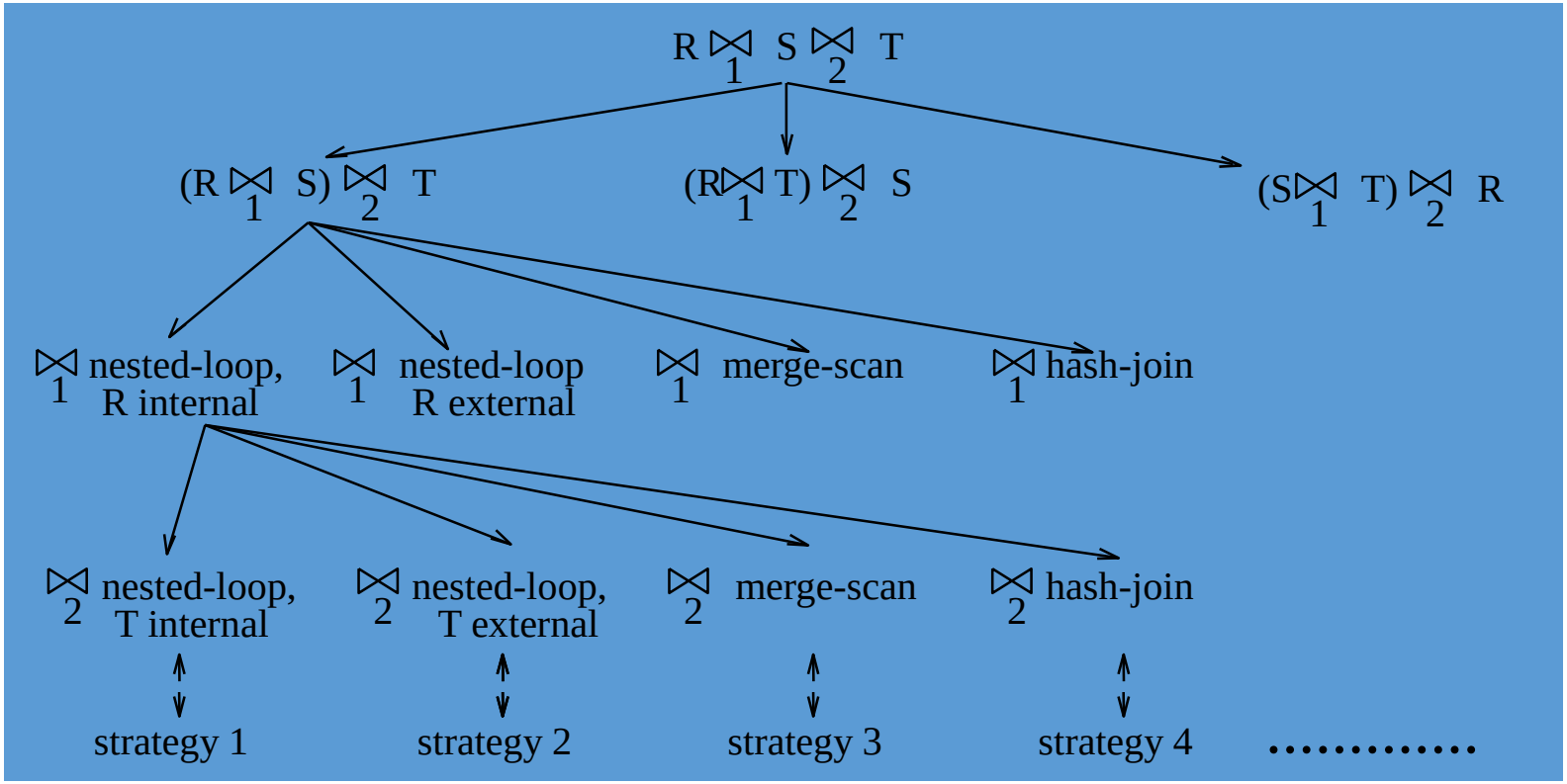
Cost-based optimization

- An optimization problem, whose decisions are:
  - The data access operations to execute
    (e.g., scan vs index access)
  - The order of operations (e.g., the join order)
  - The option to allocate to each operation
    (e.g., choosing the join method)
  - Parallelism and pipelining can improve performances

Approach to query optimization

- Optimization approach:
  - Make use of profiles and of approximate cost formulas
  - Construct a decision tree, in which
    - each node corresponds to a choice
    - each leaf node corresponds to a specific execution plan

An example of decision tree



Approach to query optimization

- Assign to each plan a cost:
  - $C_{total} = C_{I/O}\, n_{I/O} + C_{cpu}\ \ n_{cpu}$
- Choose the plan with the lowest cost, based on operations research (branch and bound)
- Optimizers should obtain ‘good’ solutions in a very short time

Approaches to query execution

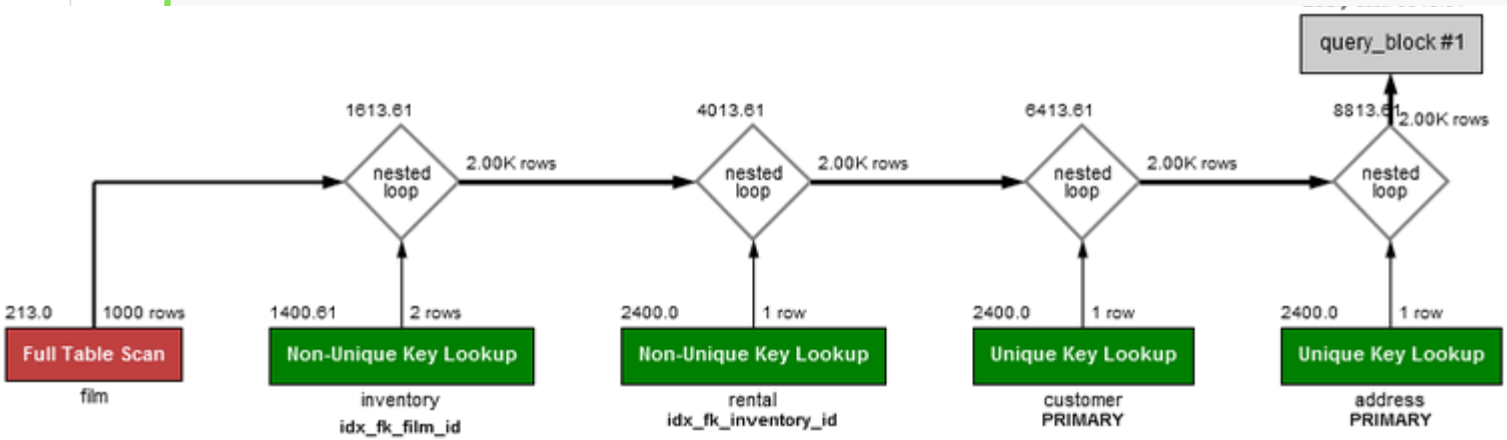
- **Compile and store**: the query is compiled once and executed many times (**prepared statement**)
  - The internal code is stored in the DBMS, together with an indication of the dependencies of the code on the particular versions of catalog used at compile time
  - On relevant changes of the catalog, the compilation of the query is invalidated and repeated
- **Compile and go**: immediate execution, no storage
  - Even if not stored, the code may live for a while in the DBMS and be available for other executions

Summary: Query Optimization

- Important task of DBMSs
- Goal is to minimize # I/O blocks
- Search space of execution plans is huge
- Heuristics based on algebraic transformation lead to good logical plan (e.g., apply first the operations that reduce the size of intermediate results), but no guarantee of optimal plan
- More details in other books (suggested: Elmasry-Navathe)

Query plan in MySQL workbench

```sql
SELECT CONCAT(customer.last_name, ', ', customer.first_name)
  AS customer, address.phone, film.title FROM rental
INNER JOIN customer ON rental.customer_id = customer.customer_id
INNER JOIN address ON customer.address_id = address.address_id
INNER JOIN inventory ON rental.inventory_id = inventory.inventory_id
INNER JOIN film ON inventory.film_id = film.film_id
WHERE rental.return_date IS NULL
AND rental_date + INTERVAL film.rental_duration DAY < CURRENT_DATE()
LIMIT 5;
```



https://dev.mysql.com/doc/workbench/en/wb-performance-explain.html

Determine the execution plan of your query

- “EXPLAIN PLAN” SQL statement
- Oracle: http://docs.oracle.com/cd/B28359_01/server.111/b28274/optimops.htm
- SQLite: http://www.sqlite.org/eqp.html
- MySQL: https://dev.mysql.com/doc/refman/8.0/en/execution-plan-information.html
- MS SQL server: http://msdn.microsoft.com/en-us/library/ms176005(v=sql.105).aspx

COSTS OF DIFFERENT ACCESS MODES

Query optimization

- We learned about tree & hash indexes
  - How does the DBMS know when to use them?
- The same query can be executed by the DBMS in many ways
  - How does the DBMS decide which option is best?

Query optimization

- Optimizer:
  - it receives a query written in SQL and
  - produces a program in an internal format that uses the data access methods
- Steps:
  - Lexical, syntactic and semantic analysis
  - Translation into an internal representation (similar to algebraic trees)
  - Algebraic optimization
  - Cost-based optimization
    - The query may be “rewritten” by the DBMS, e.g., turning joins into nested queries
  - Code generation

A simple example of query optimization

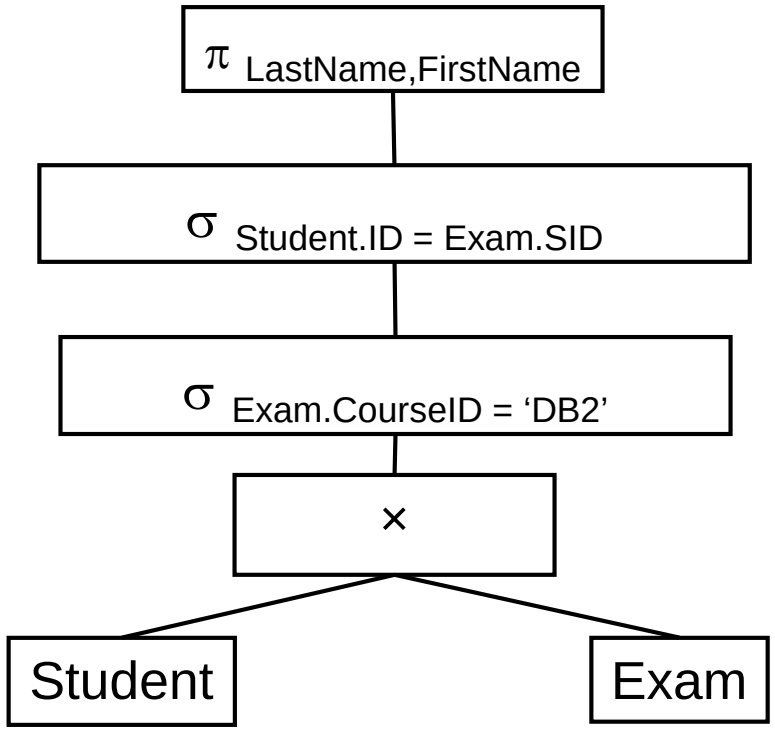
STUDENT (ID, FirstName, LastName, Email, City, Birthdate)

EXAM (SID, CourseID, Date, Grade)

- Query

```
SELECT LastName, FirstName
FROM Student, Exam
WHERE
  Student.ID = Exam.SID
AND
   Exam.CourseID = “DB2”
```

Query Tree



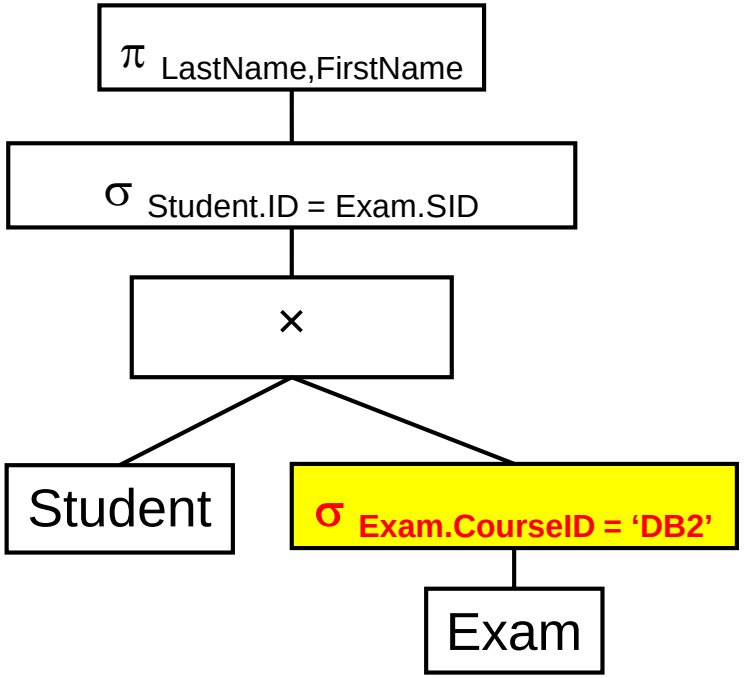

```
SELECT LastName, FirstName
FROM Student, Exam
WHERE Student.ID = Exam.SID AND
Exam.CourseID = "DB2"
```

- Many ways to optimise queries:
  - Changing the query tree to an equivalent but more efficient one
  - Exploiting database statistics
  - Choosing efficient implementations of each operator

Optimisation Example

- Equivalent query:
  - Selecting Exam entries with CourseID = ‘DB2’
  - Taking the Cartesian product of the result with Student

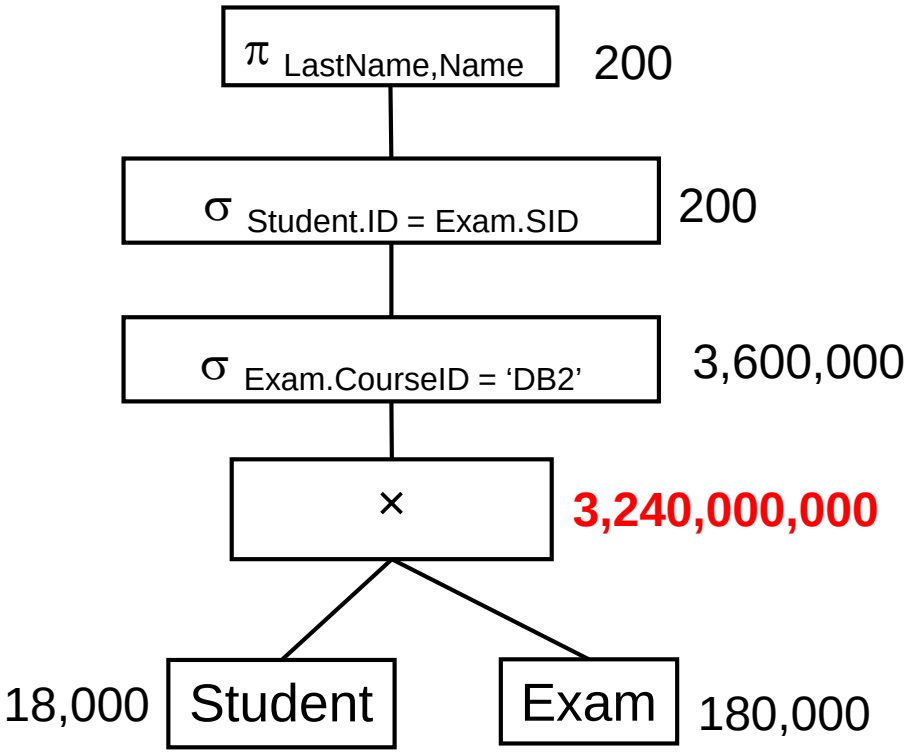
```
SELECT LastName, FirstName
FROM Student, Exam
WHERE Student.ID = Exam.SID AND
Exam.CourseID = “DB2”
```

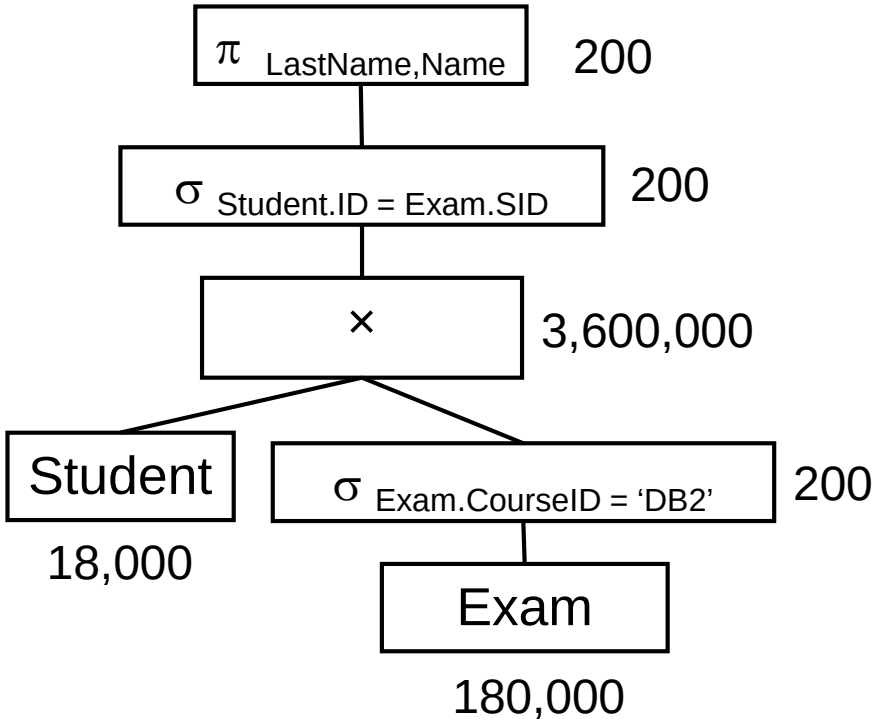


Optimisation Example

- Consider the following statistics
  - The university has 18,000 students
  - Each student is enrolled in ~10 exams
  - Only 200 take DB2

```
SELECT LastName, FirstName
FROM Student, Exam
WHERE Student.ID = Exam.SID
AND Exam.CourseID = "DB2"
```





Relation profiles

- Profiles are stored in the data dictionary and contain quantitative information about tables:
  - the cardinality (number of tuples) of each table T
    - Select count (*) from T may be executed directly on the dictionary without accessing the table data
  - the dimension in bytes of each attribute A_j in T
  - the number of distinct values of each attribute A_j in T: val(A_j)
  - the minimum and maximum values of each attribute A_j in T
    - Queries with WHERE conditions using < and > may be executed directly on the dictionary
- Periodically calculated by activating appropriate system primitives (for example, the update statistics command)
- Used in cost-based optimization for estimating the size of the intermediate results produced by the query execution plan

Data profiles and selectivity of predicates

- Selectivity = probability that any row will satisfy a predicate
  - If val(A) = N (attribute A has N values) and
  - the values are homogeneously distributed over the tuples, then
  - the selectivity of a predicate in the form A=k is 1/N
- Example: val(City)=200 → selectivity for City = 1/200 = 0.5%
  - 18K students → 90 students per city (on average)
- If no data on distributions are available, we will always assume homogeneous distributions!

Optimizations

Operations

- Selection
- Projection
- Sort
- Join
- Grouping

Access methods

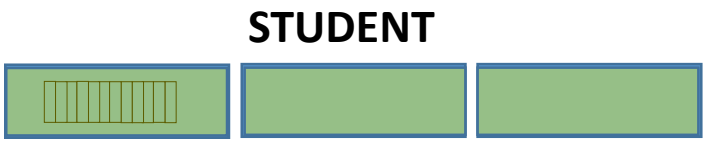
- Sequential
- Hash-based indexes
- Tree-based indexes

Running example

- Block size 8KB (8192 bytes)
- STUDENT (ID, Name, LastName, Email, City, Birthdate, Sex)
  - 150K tuples
  - Each tuple is 95 bytes long (4+20+20+20+20+10+1)
  - $t_{STUDENT}$ = 8192/95 = 87 tuples per block (block factor)
  - $b_{STUDENT}$ = 150000/87 = 1.7k blocks
- EXAM (SID, CourseID, Date, Grade)
  - 1.8 M tuples
  - Each tuple is 24 bytes long (4+4+12+4)
  - t_{EXAM} = 8192/24 = 340 tuples per block (block factor)
  - b_{EXAM} = 1800000/340 = 5.3k blocks

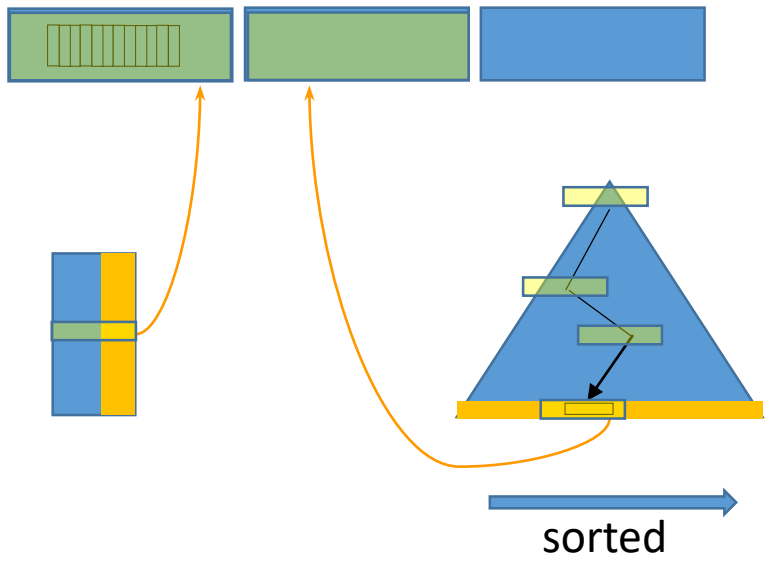
Sequential scan

- Performs a sequential access to all the tuples of a table or of an intermediate result, at the same time executing various operations, such as:
  - Projection to a set of attributes
  - Selection on a simple predicate (of type: $A_i = v$)



Hash and tree based access

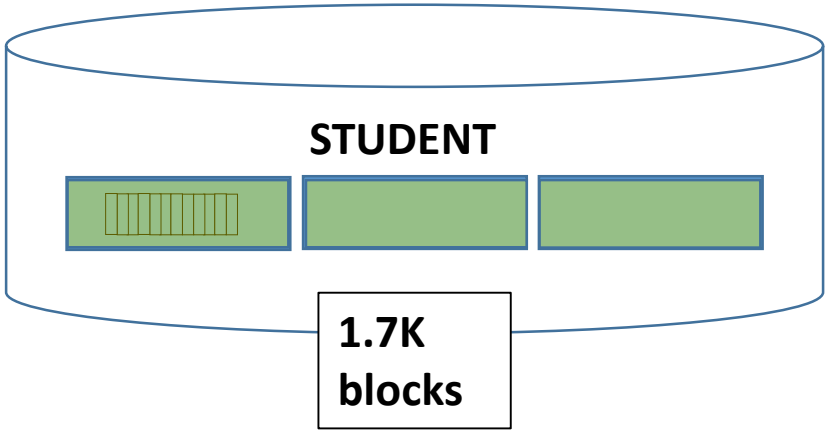
- Hash **indexes** support:
  - Selection or join with equality predicates
- Tree-based **indexes** support:
  - selection or join criteria
  - ORDER BY
  - GROUP BY
  - other operations involving sorting (such as UNION or DISTINCT)



Sequential scan: order key matters

- Example – suppose STUDENT sequentially ordered by ID:

```
SELECT *
FROM STUDENT
WHERE City = "Milan"
```



- Cost: **1.7K** I/O accesses

```
SELECT *
FROM STUDENT
WHERE ID<500
```

- Cost: **< 1.7K** I/O accesses because one can stop when ID=500 (order is by ID)

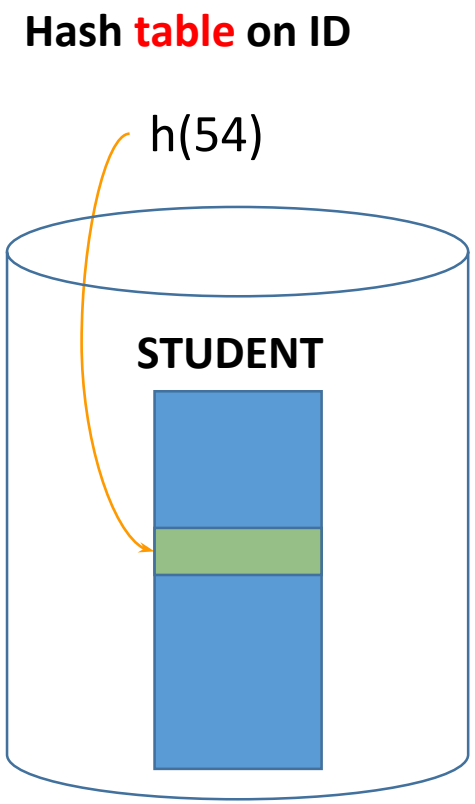
Cost of lookups: equality (A_i=v)

- Sequential structures with no index
  - Lookups are not supported (cost: a full scan)
    - Sequentially-ordered structures may have reduced cost
- Hash/Tree structures
  - Supported if A_i is the search key attribute of the structure
  - The cost depends on
    - the storage type (primary/secondary)
    - the search key type (unique/non-unique)

Equality lookup on a primary hash

```
SELECT *
FROM STUDENT
WHERE ID='54'
```

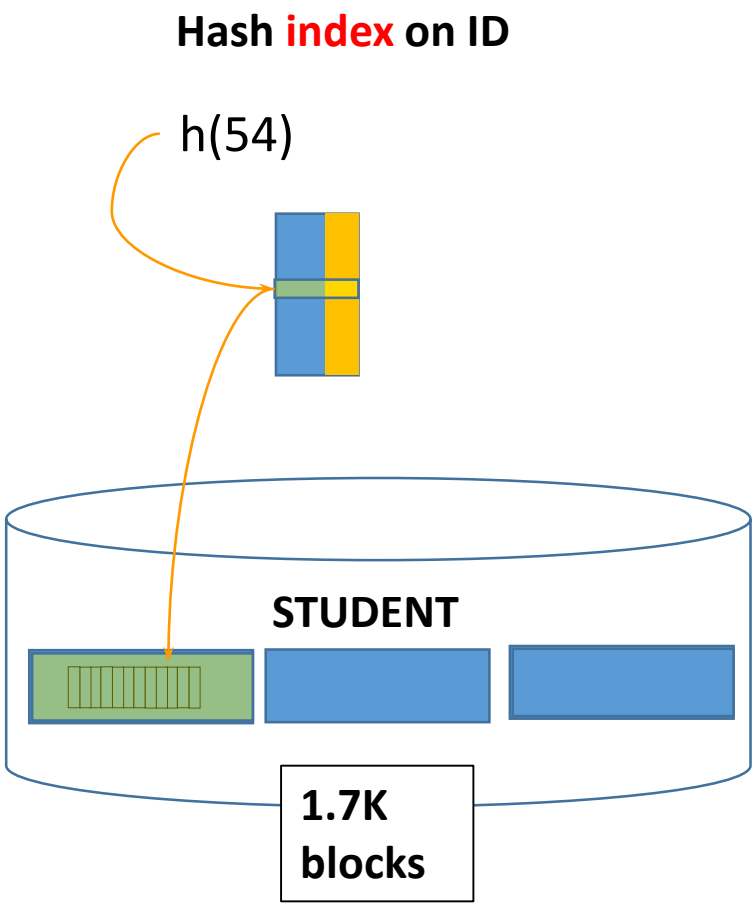
- Query predicate on search key
- Cost:
  - no overflow chains
    - cost = 1
  - With overflow chain with cost 0.3
    - cost = 1.3



Equality lookup on a secondary hash (unique search key)

```
SELECT *
FROM STUDENT
WHERE ID='54'
```

- Query predicate on search key
- Cost:
  - no overflow chains
    - cost = 1 + 1
  - With overflow chain with cost 0.3
    - cost = 1.3 + 1

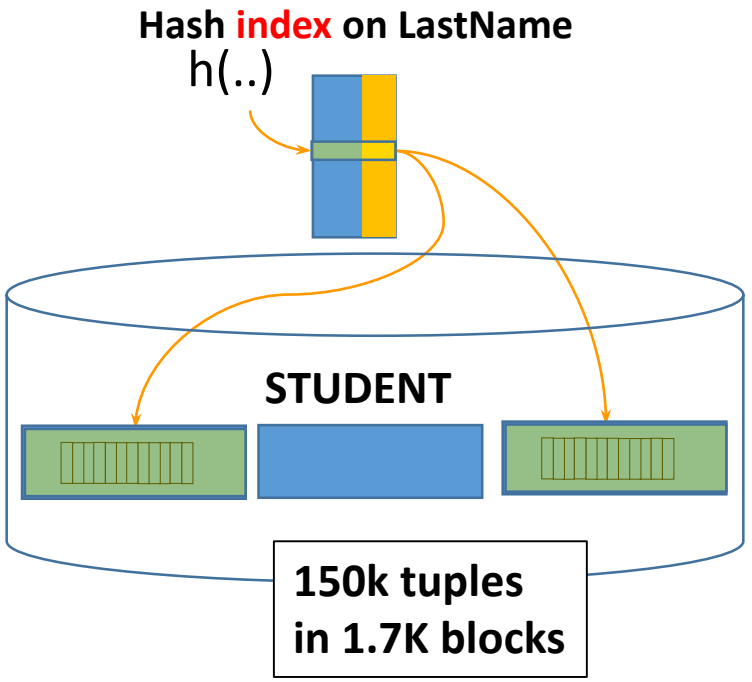


Equality lookup on a secondary hash (non unique search key)

```
SELECT *
FROM STUDENT
WHERE LastName="Rossi"
```

Note: you cannot assume that the «Rossi» tuples are in the same block of **primary** storage

- Cost with/without overflow chains:
  - 1 [+ the size of overflow chain (e.g., 0.3)]
  - + cost of accessing relevant blocks in primary storage (…how many?)
- Suppose val(LastName)=75K
  - STUDENT contains 150K tuples → on average each lastname appears twice
  - On average 2 blocks per lastname
- TOTAL COST (with overflow chain) =
  - 1.3 (cost to access the hash index) + 2 (cost to access 2 tuples of STUDENT needed for the SELECT *) = 3.3

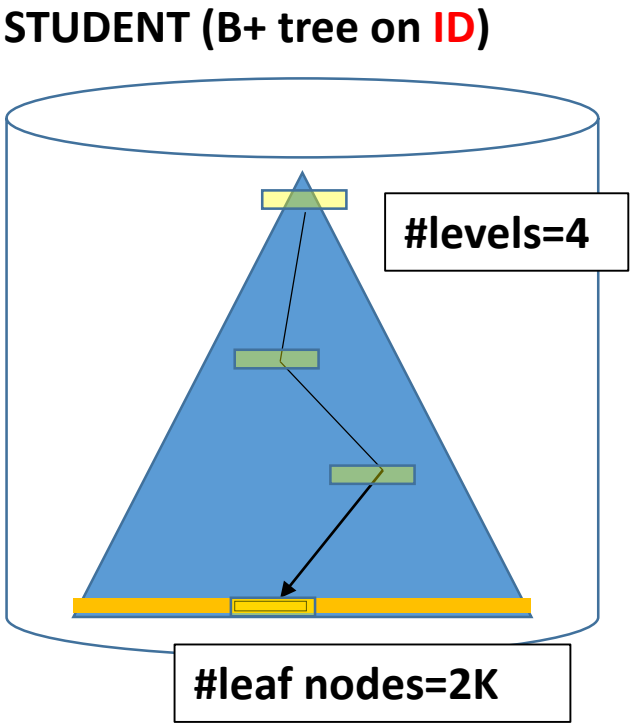


Equality lookup on a primary B+

- Query predicate **on unique** search key
  - Only 1 tuple per key value

```
SELECT *
FROM STUDENT
WHERE ID=54
```

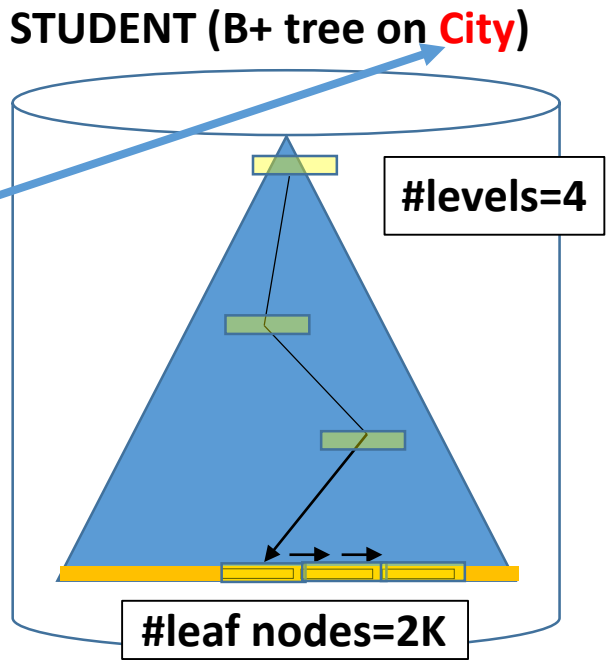
- Cost:
  - 3 intermediate levels + 1 leaf node = 4
- Remember: primary B+ leaves contain the whole tuples (seldom used..)



Equality lookup on a primary B+

- Query predicate **on non unique** search key
  - Multiple tuples per key value

```
SELECT *
FROM STUDENT
WHERE City="Milan"
```



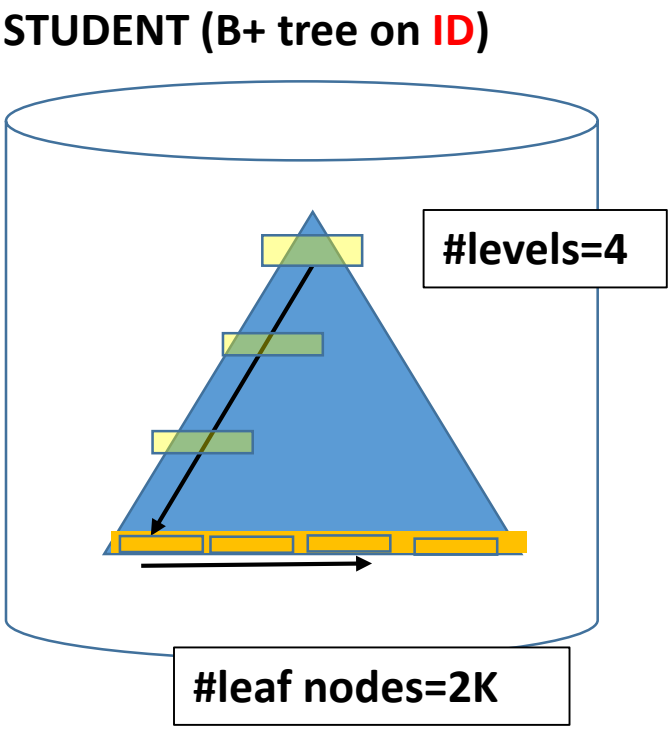
- Cost:
  - #levels to access the leaf = 4
  - How many Milan students blocks?
  - With statistics: val(City) = 150 → 150K / 150 = 1K tuples/City
  - #tuples in 1 leaf node: 150K tuples / 2K nodes = 75 tuples/node
  - #blocks with Milan tuples:
    - 1K Milan tuples / 75 tuples/node = 13.33 → 14 leaf nodes sequentially chained
  - Total cost
    - = 3 intermediate levels + 14 leaf blocks = 17

Equality lookup on a primary B+

- Query predicate **NOT** on search key

```
SELECT *
FROM STUDENT
WHERE city="Milan"
```

- All the tuples are in the leaf nodes, we need to reach & scan all of them
- Cost:
  - 3 intermediate levels + 2K leaf nodes

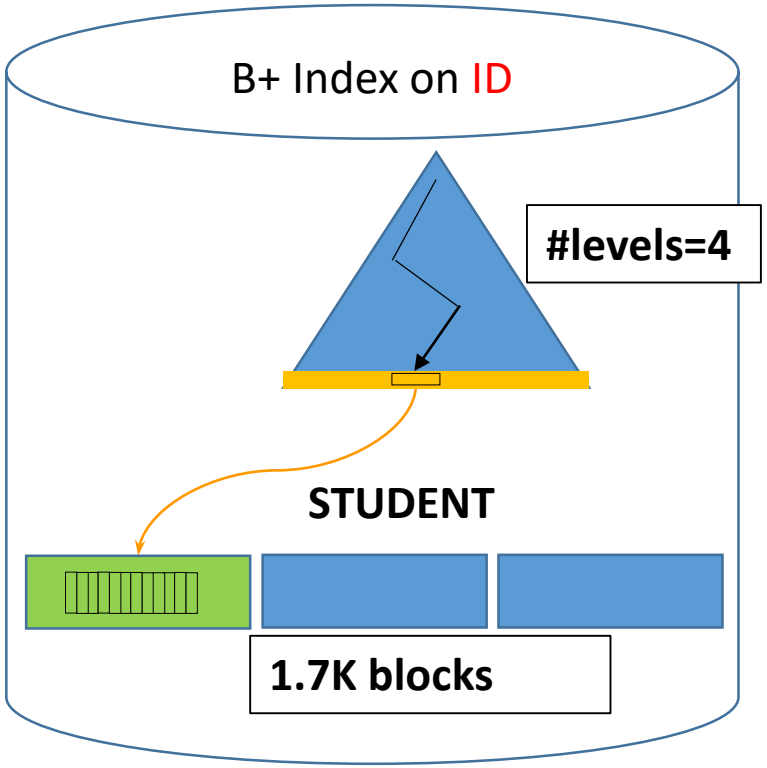


Equality lookup on secondary B+

- Query predicate **on unique** search key of STUDENT
  - Only 1 pointer per key value

```
SELECT *
  FROM STUDENT
  WHERE ID="54"
```

- Cost:
  - 3 intermediate levels + 1 leaf node + 1 data block = 5

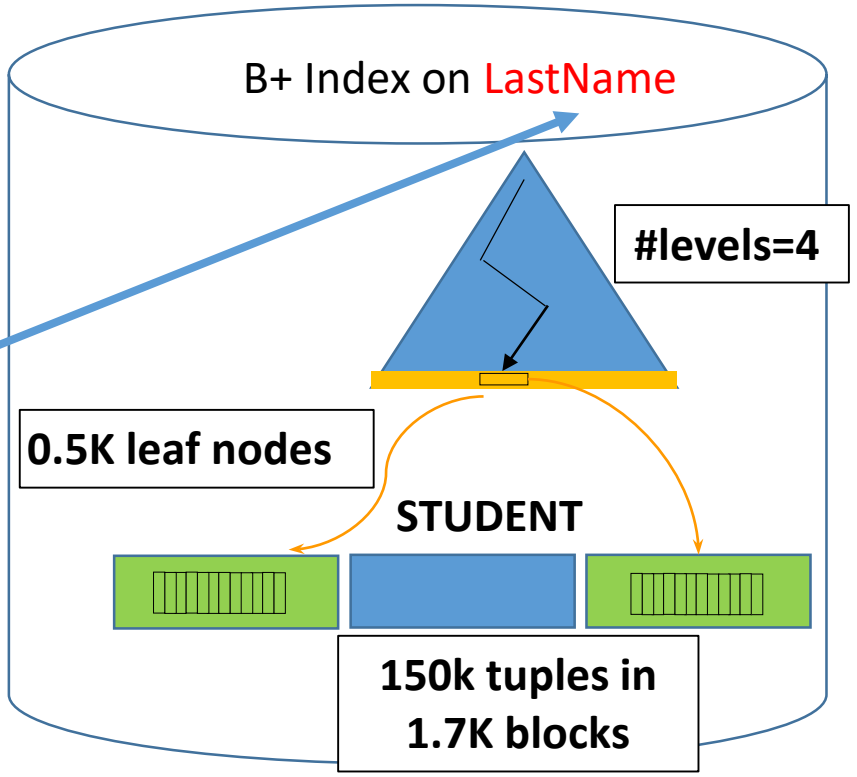


Equality lookup on secondary B+

- Query predicate **on non unique** search key of STUDENT
  - Multiple pointers per key value

```
SELECT *
FROM STUDENT
WHERE LastName="Rossi"
```

- Suppose Val(LastName) = 75K → 2 tuples / Lastname
- Cost:
  - #of levels of the B+tree: 3 + 1 (2 pointers fit in 1 leaf node, each leaf holds 150k/0,5k = 300 pointers)
  - #of blocks to be accessed in STUDENT following the B+ tree pointers in the leaf: 2
  - Total cost = 3 +1 + 2 = 6



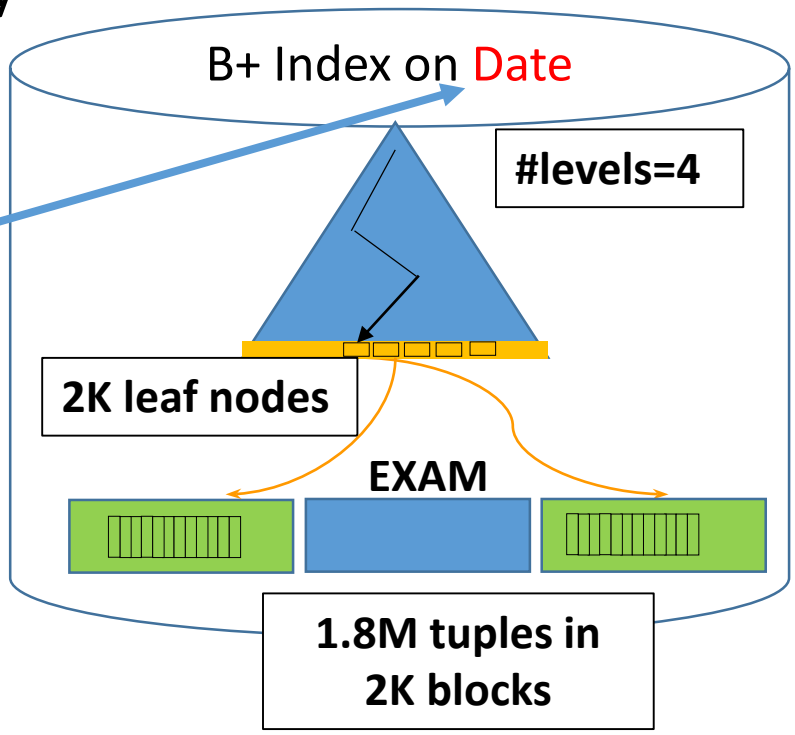
We assume that we reload a block for tuple Ti even if we have loaded it already for a tuple Tj (no caching)

Equality lookup on a **secondary** B+

- Query predicate **on non unique** search key of EXAM
  - Multiple pointers per key value

```
SELECT *
FROM EXAM
WHERE Date="10/6/2019"
```

- Suppose val(Date) = 500
  → 1.8M / 500 = 3.6K tuples/date
- Cost factors =
  - #of intermediate levels **3**
  - pointers per leaf **1.8M / 2K = 900**
  - leaf nodes per date: ***3.6K / 900 = 4*** *(but in most of the cases, the given date may not be the left-most key in the block → in average the 900 pointers will be in **5** blocks)*
  - blocks to access in EXAM (1 per pointer) = **3.6K**
- **Total cost = 3 + 5 + 3.6K = ~ 3.6K**
  - [Note that full scan costs less]



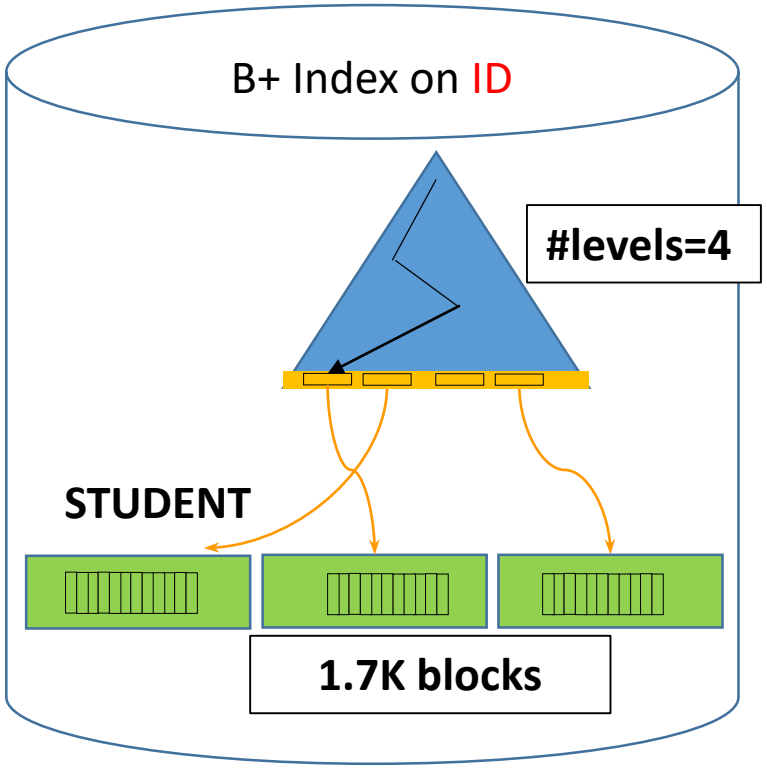
We assume that we reload a block for tuple Ti even if we have loaded it already for a tuple Tj (no caching)

Equality lookup on secondary B+

- Query predicate **not on** search key of STUDENT

```
SELECT *
FROM STUDENT
WHERE city="Milan"
```

- Cost:
  - Index is not useful for this query (full scan costs 1.7k)



Interval lookups $A_i<v$, $v_1<= A_i <= v_2$

- Sequential structures (primary)
  - Lookups are not supported (cost: a full scan)
    - Sequentially-ordered structures may have reduced cost
- Hash structures (primary/secondary)
  - Lookups based on intervals are not supported
- Tree structures (primary/secondary)
  - Supported if A_i is the search key
  - The cost depends on
    - the storage type (primary/secondary) – see next slides
    - the index key type (unique/non-unique)

Interval lookup on **primary** B+

- We consider a lookup for $v_1 <= A_i <= v_2$ as the general case
  - If $A_i<v$ or $v<A_i$ we just assume that the other edge of the interval is the first/last value in the structure
- The root is read first
  - then, a node per intermediate level is read, until…
- …the first leaf node is reached, that stores tuples with $A_i = v_1$
  - If the searched tuples are all stored in that leaf block, stop
  - Else, continue in the leaf blocks chain until v_2 is reached
- Cost: 1 block per intermediate level + as many leaf blocks as necessary to read all the tuples in the interval (estimated with statistics)

Interval lookup on **secondary** B+

- We still consider a lookup for for $v_1 <= A_i <= v_2$ as the general case
- The root is read first
  - then, a node per intermediate level is read, until…
- …the first leaf node is reached, that stores the pointers pointing to the blocks containing the tuples with $A_i = v_1$
  - If all the pointers (up to v_2) are in that leaf block, stop
  - Else, continue in the leaf blocks chain until v2 is reached
- Cost: 1 block per intermediate level + as many leaf blocks as necessary to read all pointers in the interval + 1 block per each such pointer (to retrieve the tuples)

Conjunction / disjunction

- **Predicates in conjunction**

```
SELECT * FROM STUDENT
WHERE City="Milan" AND Birthdate="12/10/2000"
```

  - If supported by indexes, the DBMS chooses the **most selective** supported predicate for the data access, and evaluates the other predicates in main memory
- **Predicates in disjunction**:

```
SELECT * FROM STUDENT
WHERE City="Milan" OR Birthdate="12/10/2000"
```

  - if any of the predicates is not supported by indexes, then a scan is needed
  - if **all** are supported, indexes can be used to evaluate all predicates and then duplicate elimination is normally required

Join Methods

- Joins are the most frequent (and costly) operations in DBMSs
- There are several join strategies, among which:
  - nested-loop, merge-scan and hashed
  - These three join methods are based on scanning, ordering and hashing
- The “best” strategy is chosen based on various aspects…

Running example

- Block size 8KB (8192 bytes)
- STUDENT (ID, Name, LastName, Email, City, Birthdate, Sex)
  - 150K tuples
  - Each tuple is 95 bytes long (4+20+20+20+20+10+1)
  - $t_{STUDENT}$ = 8192/95 = 87 tuples per block (block factor)
  - $b_{STUDENT}$ = 150000/87 = 1.7k blocks
- EXAM (SID, CourseID, Date, Grade)
  - 1.8 M tuples
  - Each tuple is 24 bytes long (4+4+12+4)
  - t_{EXAM} = 340 tuples per block (block factor)
  - b_{EXAM} = 5.3k blocks

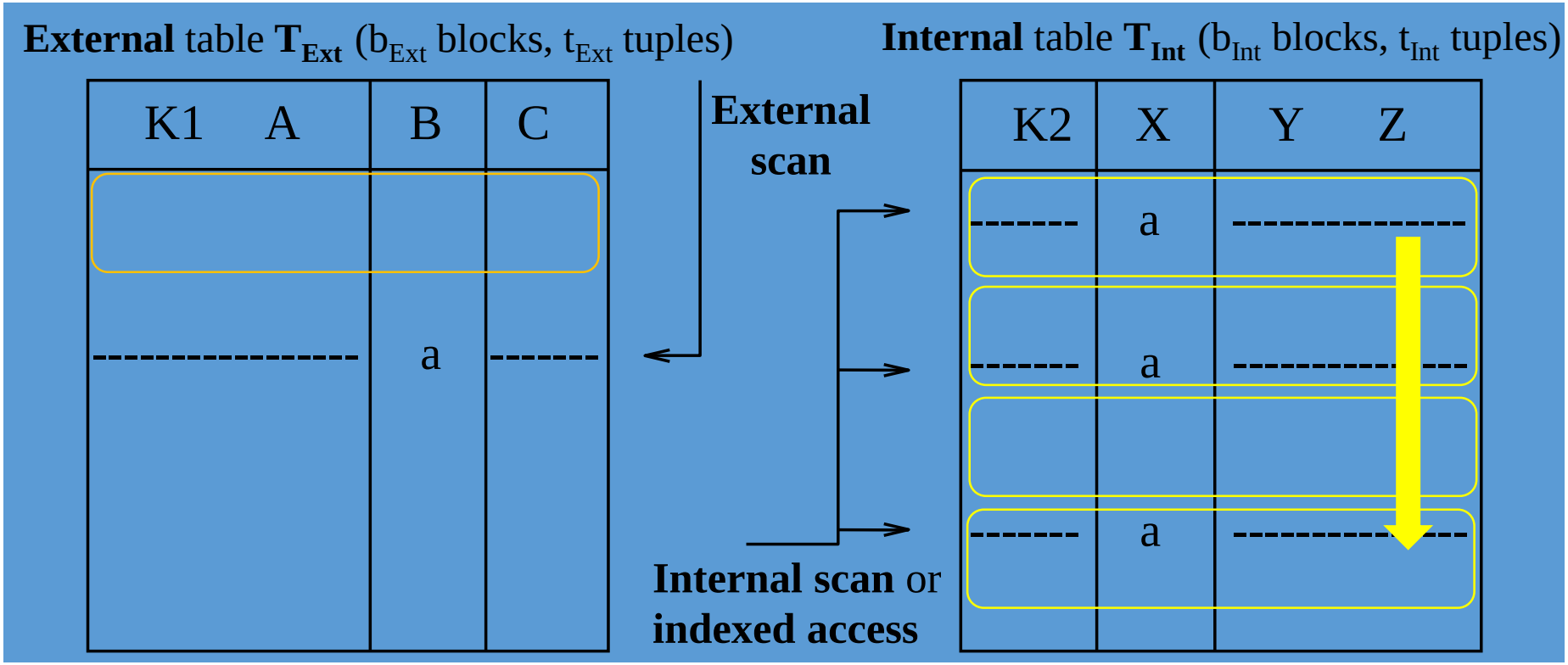
Equality Joins With One Join Column

```
SELECT  *
FROM Student, Exam
WHERE ID=SID
```

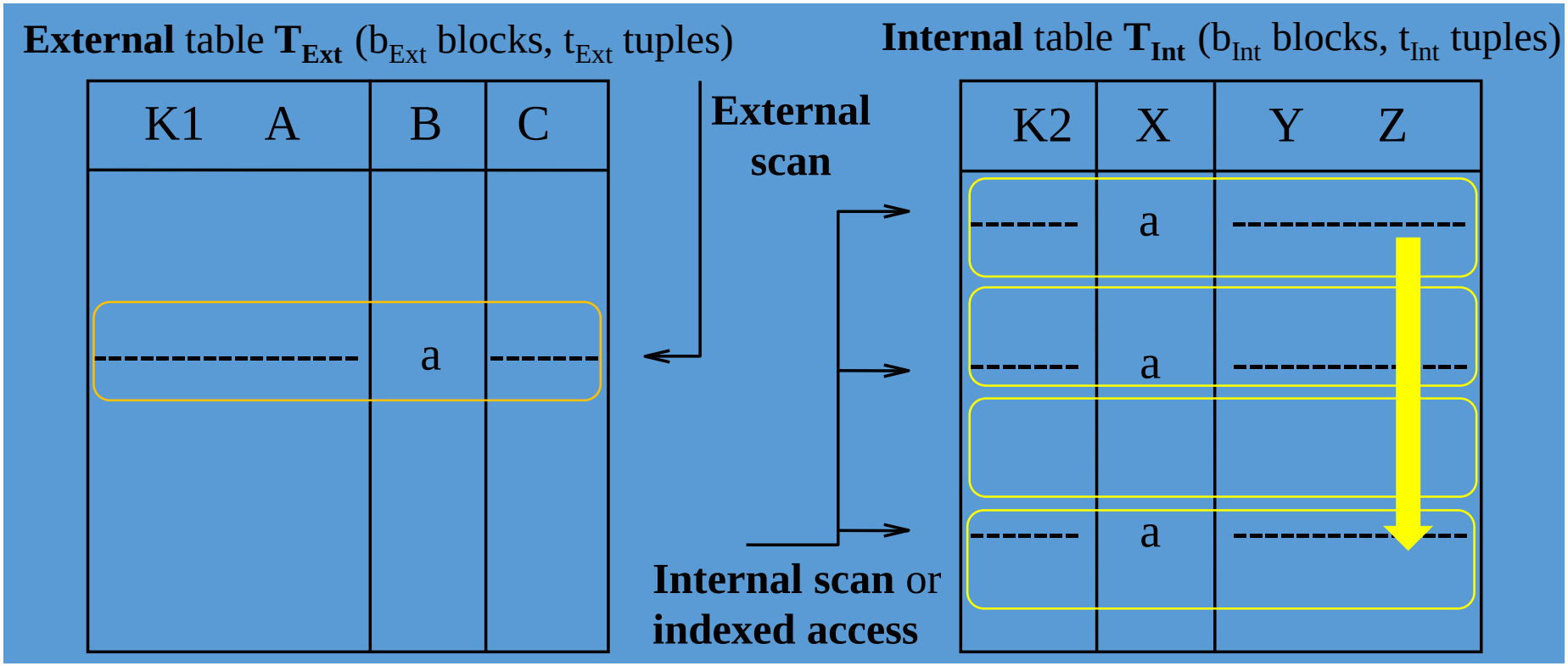
- In algebra: Student $\bowtie_{ID=SID}$ Exam
  - Common! Must be carefully optimized
  - Student $\times$ Exam is large → Student $\times$ Exam followed by a selection is inefficient

Nested-Loop join

- Scan the external table and for each block scan the internal table



Nested-Loop join



Cost of simple nested-loop joins

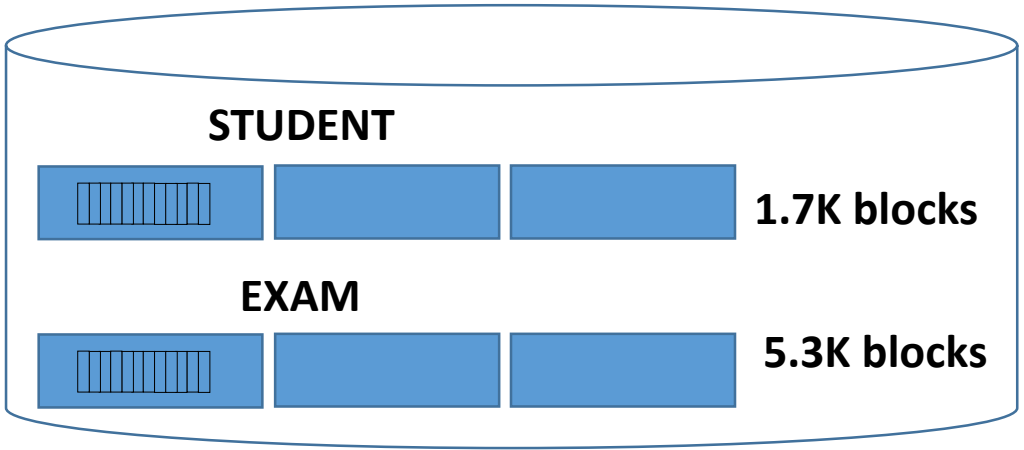
- A nested loop join compares the tuples of a block of table T_{Ext} with all the tuples of all the blocks of T_{Int} before moving to the next block of T_{Ext}
  - We always assume that the buffer does not have enough available free pages to host more than a few blocks
  - The cost is **quadratic** in the size of the tables: scan the external table and then scan the internal table once for each block of the external one
    - Cost = $b_{Ext} + b_{Ext} \times b_{Int} = b_{Ext} \times (1 + b_{Int}) =\sim b_{Ext} \times b_{Int}$
  - **Cached nested loop:** if one of the tables is small enough to fit in the buffer, then it is chosen as internal table and scanned **only once**.
    - Cost = $b_{Ext} + b_{Int}$

A simple Nested-Loop join

- STUDENT (Id, Name, LastName, Email, City, Birthdate, Sex)
- EXAM (SID, CourseID, Date, Grade)

```
SELECT STUDENT.*
FROM STUDENT JOIN EXAM ON ID=SID
```

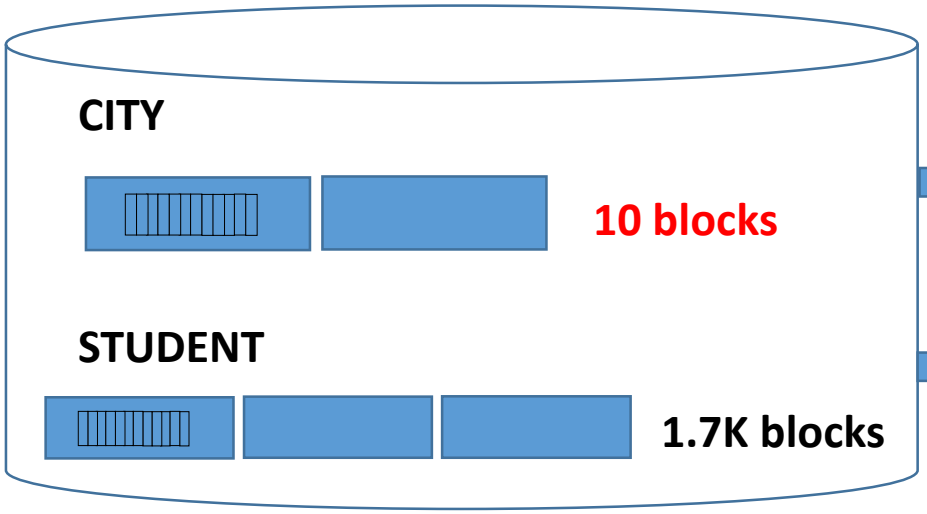
- Student external table, Exam internal table
  - Cost = 1.7K + 1.7K*5.3K =~ 9M I/O accesses
- Exam external table,Student internal table
  - Cost = 5.3K + 5.3K*1.7K =~ 9M I/O accesses



Nested-Loop join with cache

- STUDENT (ID, Name, LastName, Email, City, Birthdate, Sex)
- CITIES (City, Region, Description)
  - b_{CITIES} = 10 blocks, with block size 8KB → 80 KB → cacheable

`SELECT STUDENT.* FROM STUDENT NATURAL JOIN City`



Cache CITIES (10 blocks)
Cost = **10** I/O accesses

For all the 1.7K blocks of STUDENT, compute the join of their tuples with the cached tuples
Cost=**1.7K** I/O accesses for STUDENT

Total cost = 10 + 1,7K =~ **1,7K**

Filtering by condition: option 1

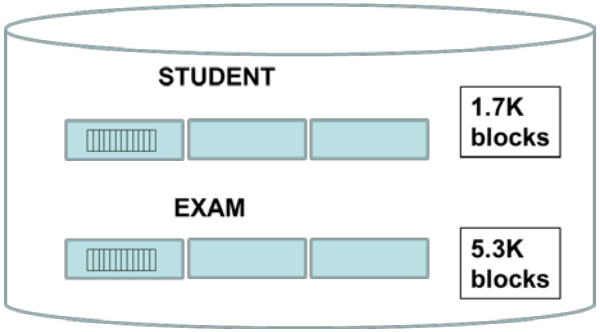
- STUDENT (ID, Name, LastName, Email, City, Birthdate, Sex)
- EXAM (SID, CourseID, Date, Grade)

```
SELECT STUDENT.*
FROM STUDENT JOIN EXAM ON ID=SID
WHERE City='Milan' AND Grade='30'
```

val(City) = 150
val(Grade) = 17

Option 1: Scan Student, evaluate City='Milan' + Lookup on Exam

- Read all the STUDENT blocks: **1.7K**
- **Filter** students from Milan → 150K tuples / 150 cities = **1k**
- For 1k students do the join with the tuples of all the 5.3K blocks in EXAM → this requires 1k scans



Total cost:
1,7K + 1K*5.3K
=~ **5.3M** I/O accesses

Filtering by condition: option 2

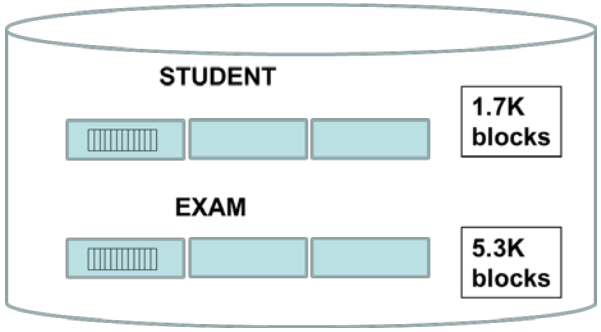
- STUDENT (ID, Name, LastName, Email, City, Birthdate, Sex)
- EXAM (SID, CourseID, Date, Grade)

```
SELECT STUDENT.*
FROM STUDENT JOIN EXAM ON ID=SID
WHERE City='Milan' AND Grade='30'
```

val(City) = 150
val(Grade) = 17

Option 2: Scan Exam, evaluate Grade='30' + Lookup on Student

- Read all the EXAM blocks: **5.3K**
- **Filter** Exams with Grade='30' → 1.8M/17 different grades = **106K**
- For 106K exams do the join with 1.7K blocks in STUDENT



Total cost:
5,3K + **106K***1.7K
=~ **180M** I/O accesses

Scan and lookup (indexed nested loop)

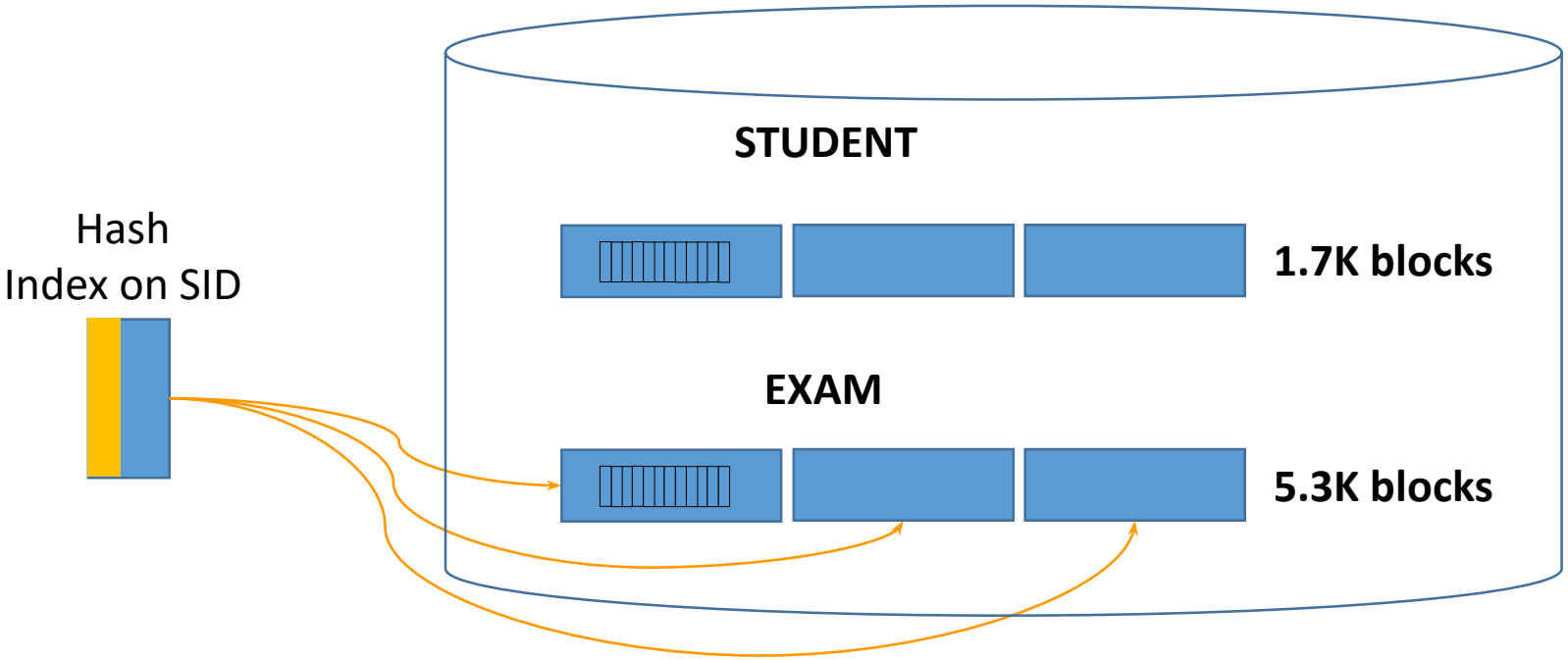
- IF one table supports **indexed access in the form of a lookup based on the join predicate**, then this table can be chosen as **internal**, exploiting the predicate to extract the joining tuples without scanning the entire table
  - If both tables support lookup on the join predicate, the one for which the predicate is more selective is chosen as internal
- Cost:
  - full scan of the blocks of the external table +
  - cost of lookup for each tuple of the external table onto the internal one, to extract the matching tuples
    - Cost = $b_{Ext} + t_{Ext} \times cost_of_one_indexed_access_to_T_{Int}$
  - NOTE: as before, if the query has a filtering predicate on the external table then t_{Ext} is the subset of the tuples of the external table that satisfy the predicate (estimated)

Scan & lookup exploiting an index

- STUDENT (ID, Name, LastName, Email, City, Birthdate, Sex)
- EXAM (SID, CourseID, Date, Grade)

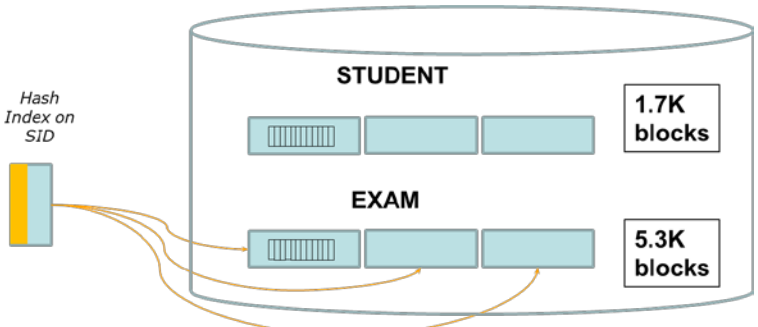
```
SELECT STUDENT.*
FROM STUDENT JOIN EXAM ON ID=SID
WHERE City='Milan' AND Grade='30'
```

val(City)=150



Filter + lookup: option 1 revised

- Improve option 1 (STUDENT as external table):
  - Scan Student, evaluate City='Milan' + Lookup on Exam **exploiting the secondary hash index on EXAM**
- Scan all the STUDENT blocks 1.7K
- Filter students from Milan -> 150K tuples / 150 different cities = 1k
- Lookup with the hash index: for 1k students
  - 1 access to the hash index (without overflow chain)
  - How many exams per student? 1.8M exams / 150K students = 12
  - Follow the 12 pointers to retrieve the exams (and the **grades needed to evaluate the where condition**)



Hash index access for Milan students

Student scan

Exam access for Milan students

Total cost =

1.7K + 1K + 1k*12 =

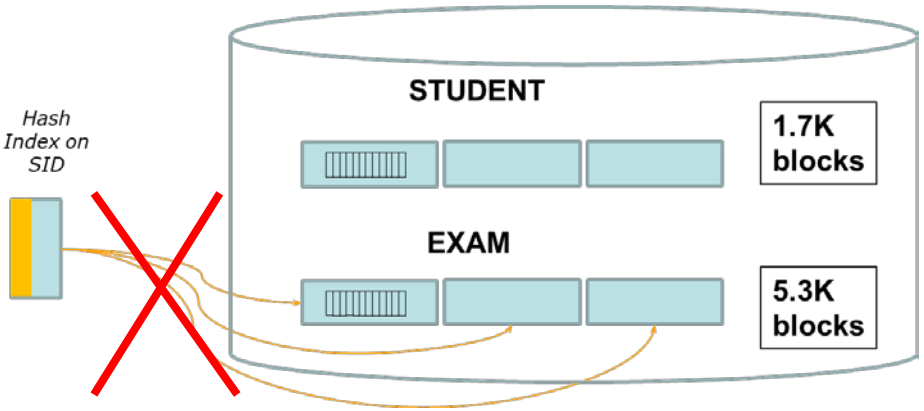
1.7K + 1K * (1+ 12) = 14.7K

Existence checking lookup

```
SELECT STUDENT.*
FROM STUDENT JOIN EXAM ON Id=SID
WHERE City='Milan' AND Grade='30'
```

val(City)=150

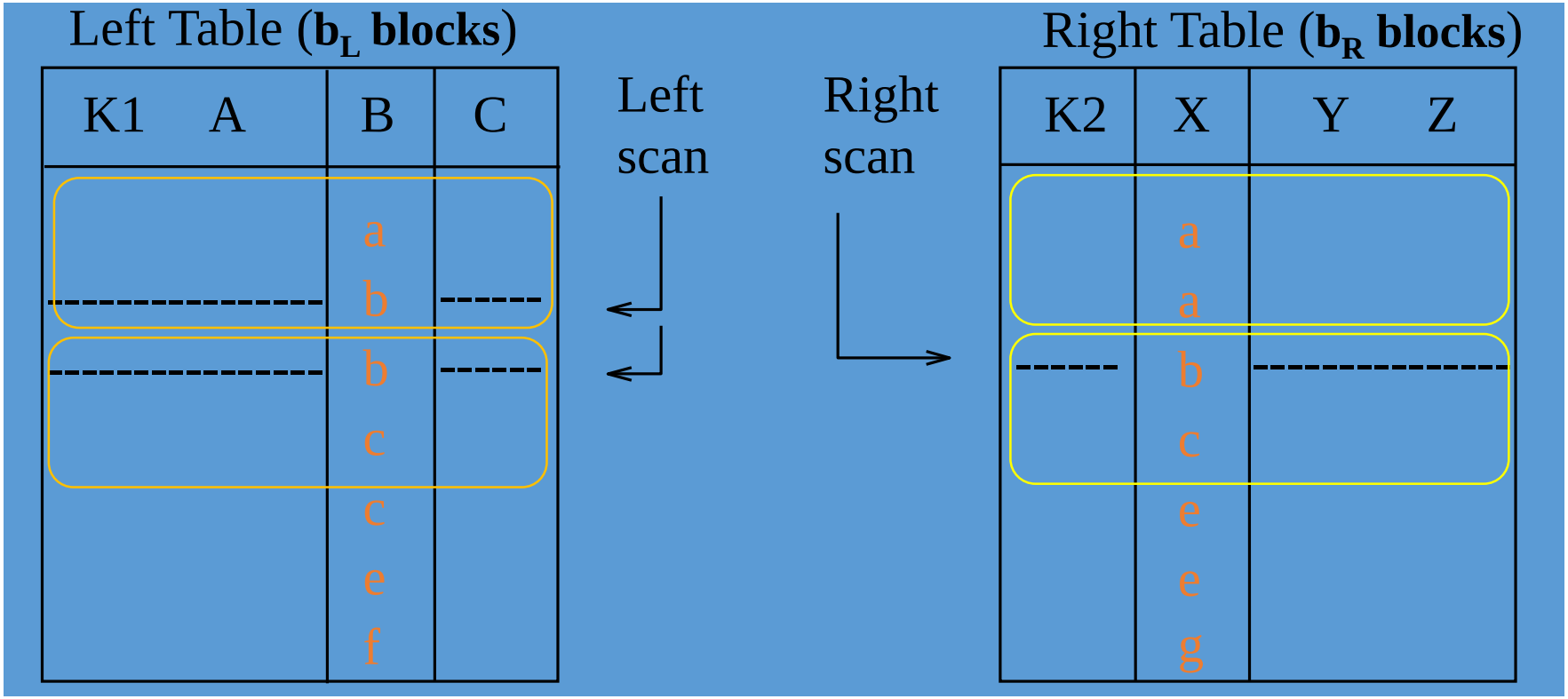
- Read all the STUDENT blocks **1.7K**
- Filter students from Milan → 150K tuples/150 different cities = **1k**
- For 1k students use the hash index: for each student
  - access to the hash index (without overflow chain): **1**
- No need to access the full EXAM tuples to do the join, only check that the joined tuple pair exists



Total cost =
1.7K + 1K * (1 + 1) = 2.7K

Merge-scan join

- This join is possible only if both tables are ordered according to the same key attribute, that is also the attribute used in the join predicate $L \underset{B=X}{\bowtie} R$



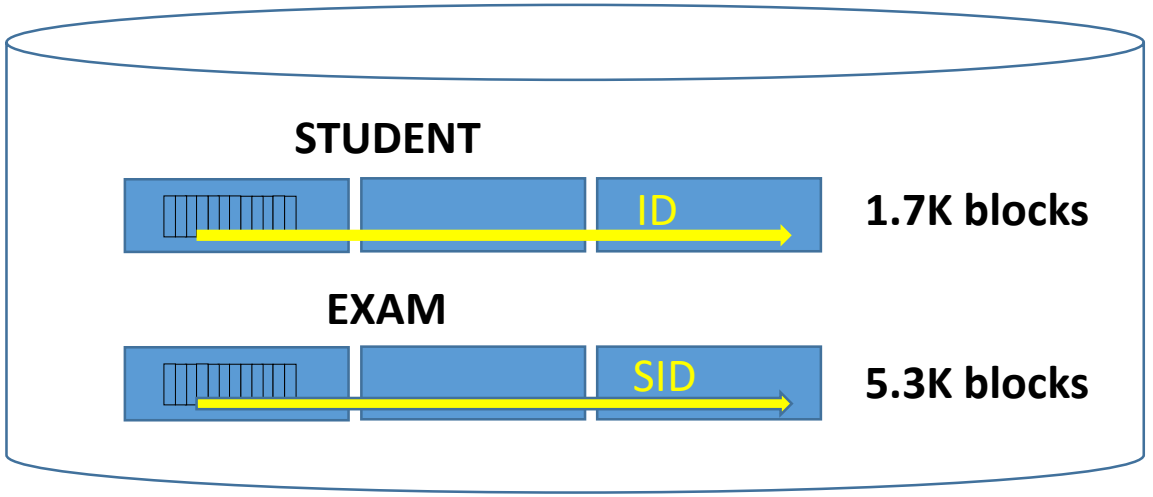
Sort + merge-scan join

- If not sorted, sort L and R **on the join column**
- Scan them to do a "merge", advancing on the tables with the least value of the join attribute
- Output result tuples <l, r>
- The cost is **linear in the size of the tables**
  - **$C = b_L + b_R$**
- The ordered full scan is possible for a table **if the primary storage is sequentially ordered wrt the join attribute or a B+ on the join attribute is defined**
- If a table needs to be sorted, add also the cost for sorting
  - $2\, b_L$ * (# of passes)
  - $2\, b_R$ * (# of passes)
    - # of passes cost = $1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil$, where N = # of blocks, B= # of buffer pages

Merge scan join on ordered tables

```
SELECT STUDENT.*
FROM STUDENT JOIN EXAM ON ID=SID
```

- Suppose
  - STUDENT: Sequentially-ordered by ID
  - EXAM: Sequentially-ordered by SID

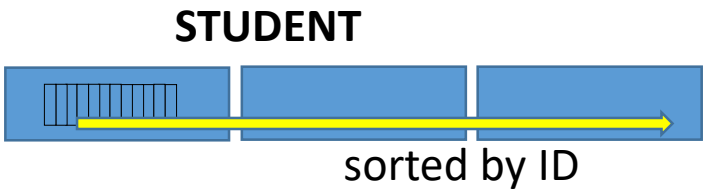


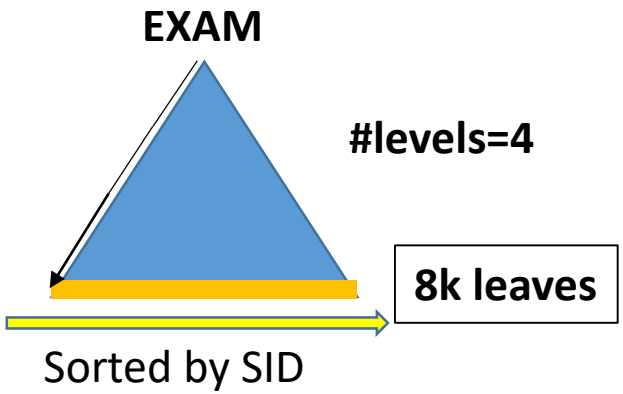
$C = b_{Student} + b_{Exam} = 1.7K + 5.3K = \mathbf{7K}$ I/Os

Indexed merge scan join

```
SELECT STUDENT.*
FROM STUDENT JOIN EXAM ON ID=SID
```

- Suppose
  - STUDENT: Sequentially-ordered by ID
  - EXAM: **primary** storage: B+ on SID (e.g., with 8k leaves)
- $C = b_{Student} + \#intermediateNodes_{Exam} + \#LeafNodes_{Exam}$
- $C = 1.7k + 3 + 8k = \sim 9.7k$

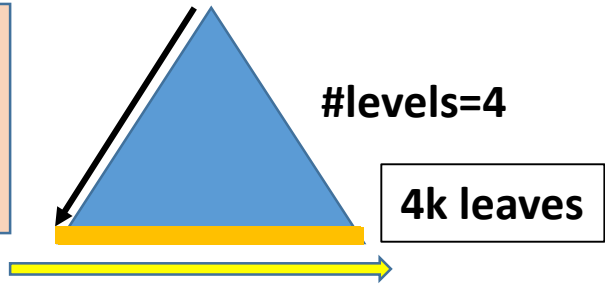


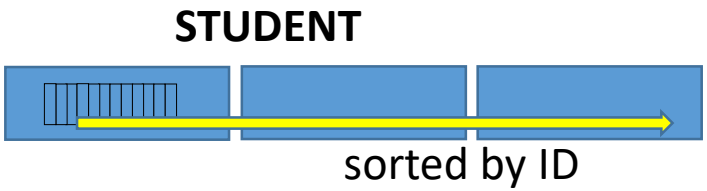


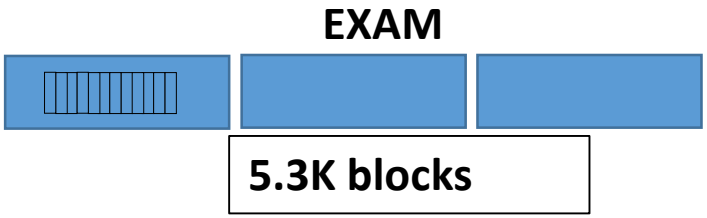
Indexed merge scan join

```
SELECT STUDENT.*
FROM STUDENT JOIN EXAM ON ID=SID
```

- Suppose
  - STUDENT: Sequentially-ordered by ID
  - EXAM: **secondary** index: B+ on SID
- $C = b_{Student} + \#intermediateNodes_{Exam} + \#LeafNodes_{Exam}$
- $C = 1.7k + 3 + 4k =$ **~ 5.7k**
- NOTE:
  - Exam tuples are not needed, so B+ leaves are sufficient
  - One pointer <--> one tuple (SID is the PK)



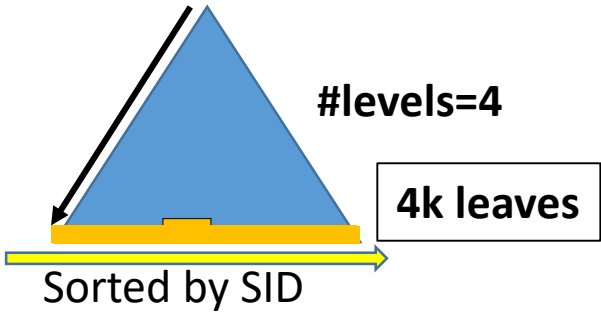


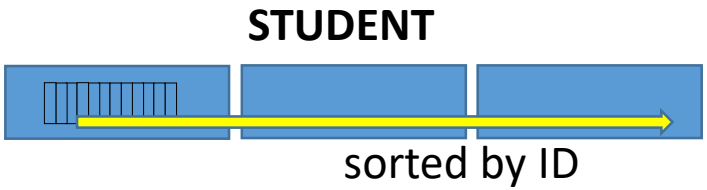


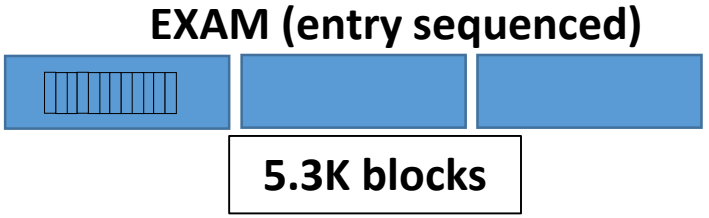
Indexed merge scan join

```
SELECT S.SID
FROM STUDENT S JOIN EXAM E ON ID=SID
WHERE GRADE>17 -- exam tuples must be accessed too
```

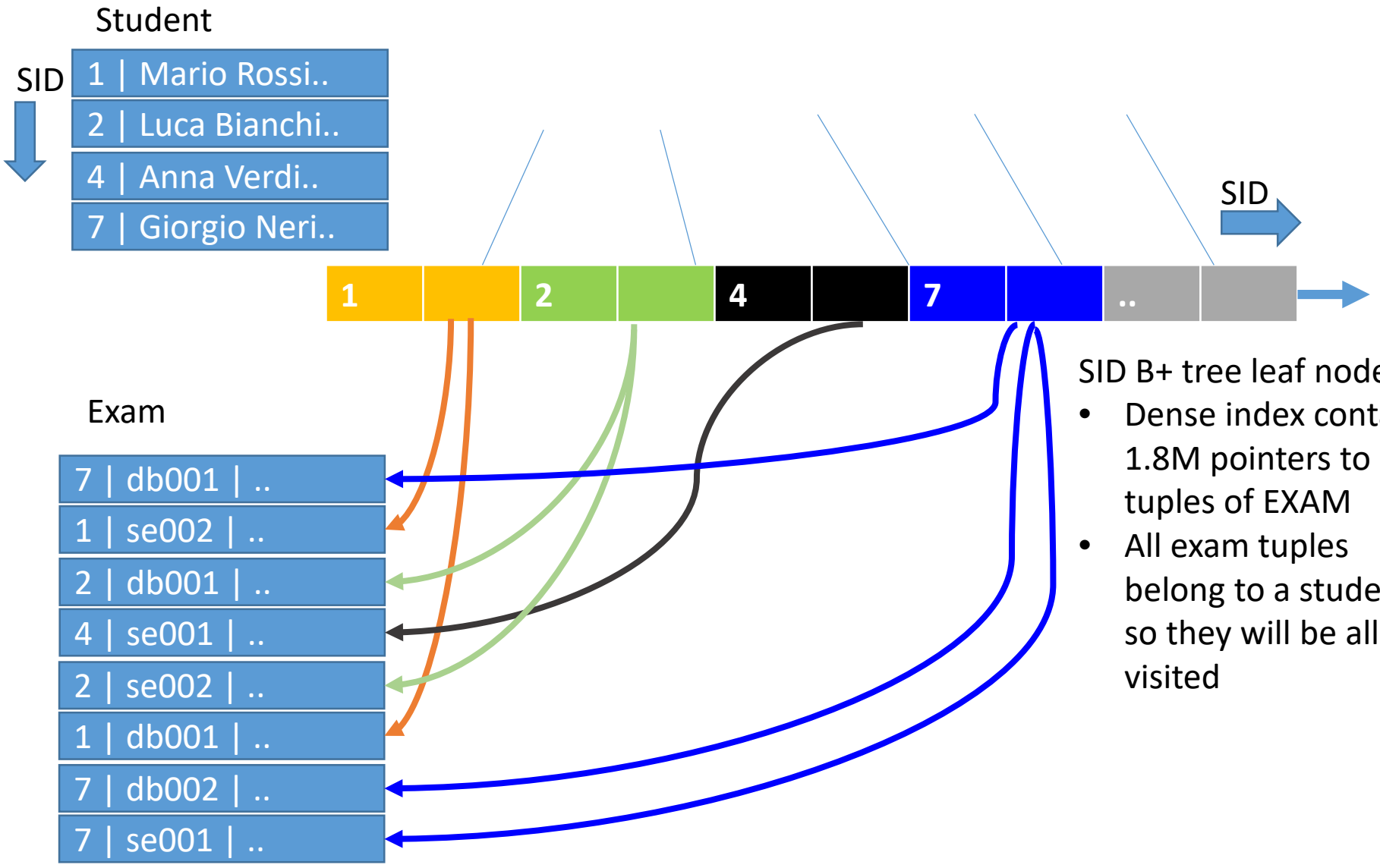
- Suppose
  - STUDENT: Sequentially-ordered by ID
  - EXAM: **secondary** index: B+ on SID
  - No statistics on grades
- C = $b_{Student}$ + #intermediateNodes + $\#LeafNodes_{Exam}$ + $\mathbf{\#Tuples}_{Exam}$
- C = 1.7k + 3 + 4k + 1.8M= **~1.8M**







Why 1.8M pointers to follow

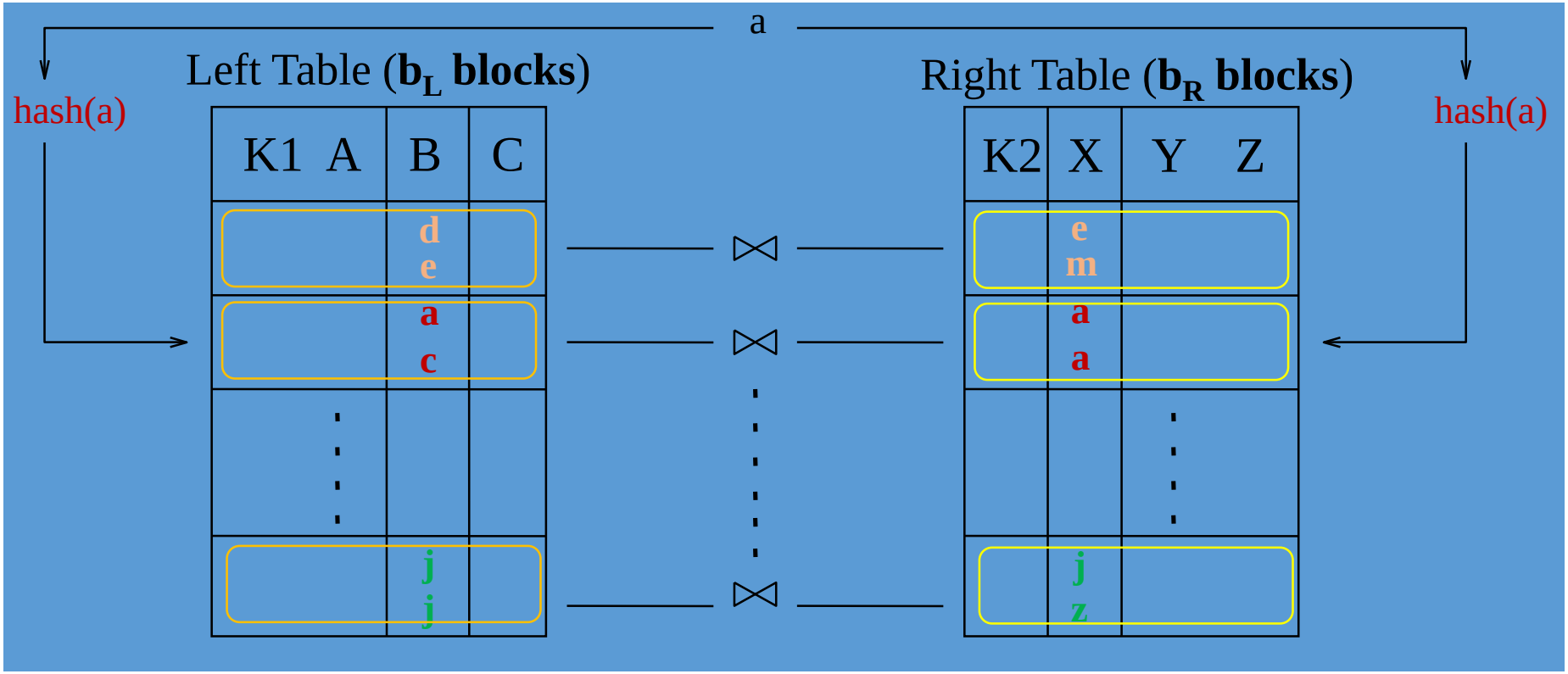


SID B+ tree leaf nodes

- Dense index contains 1.8M pointers to all tuples of EXAM
- All exam tuples belong to a student so they will be all visited

Hashed join

- This join is possible only if both tables are hashed on the same key (join) attribute with the same hash function
- Find the first/next key value X in the left table, use the hashed access to fetch the tuples with key = X from left and right table, add to the result the joined tuples that match



Cost of hashed joins

- The cost is **linear in the number of blocks of the hash-based structure**
  - If the two hashes are both primary storages:

    $C = b_L + b_R$
  - Note that the two hashes have the same number of buckets, but the number of blocks b_L and b_R may (slightly) differ due to overflows